LittleHeart: source code

UnivaBio 2026 · AI for Human Health

Project structure

ml/download_data.py: Downloads the CirCor DigiScope dataset from PhysioNet's open-data mirror.
ml/features.py: Turns a recording into a log-mel spectrogram and 4-second windows.
ml/train.py: Cross-validated training of the CNN and baseline, ONNX export, demo cases.
ml/heartbeat_check.py: Safety check calibration: refuse recordings without a heartbeat rhythm.
web/index.html: Page structure: screening, validation and education tabs.
web/js/features.js: JavaScript port of features.py, so the browser computes identical spectrograms.
web/js/model.js: Runs the ONNX model in the browser and applies the referral rule.
web/js/app.js: User interface: cases, uploads, microphone, spectrograms, results, validation charts.
web/css/style.css: Visual design, light/dark themes and phone layout.
tests/test_pipeline.py: Automated tests, including Python vs JavaScript feature parity.
tests/js_logmel.cjs: Node.js helper used by the parity test.
docs/make_pdfs.py: Generates this PDF and the one-page description.
Generated by ml/train.py (not listed): web/model/murmur_cnn.onnx (trained network), config.json (spectrogram settings and thresholds), metrics.json/.js (validation results), web/samples/*.wav (demo recordings from the dataset).

ml/download_data.py

Downloads the CirCor DigiScope dataset from PhysioNet's open-data mirror.

```
"""
Downloads the CirCor DigiScope heart sound dataset (PhysioNet, CC BY 4.0)
from PhysioNet's open-data mirror on AWS, which is much faster than the zip.

Run:  python ml/download_data.py
"""

import urllib.parse
import urllib.request
import xml.etree.ElementTree as ET
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

BUCKET = "https://physionet-open.s3.amazonaws.com"
PREFIX = "circor-heart-sound/1.0.3/"
OUT = Path(__file__).resolve().parent.parent / "data" / "circor-1.0.3"
NS = "{http://s3.amazonaws.com/doc/2006-03-01/}"


def list_keys():
    keys, token = [], None
    while True:
        url = f"{BUCKET}/?list-type=2&prefix={PREFIX}"
        if token:
            url += "&continuation-token=" + urllib.parse.quote(token)
        root = ET.fromstring(urllib.request.urlopen(url).read())
        keys += [c.find(NS + "Key").text for c in root.iter(NS + "Contents")]
        token_el = root.find(NS + "NextContinuationToken")
        if token_el is None:
            return keys
        token = token_el.text


def fetch(key):
    target = OUT / key[len(PREFIX):]
    if target.exists() and target.stat().st_size > 0:
        return
    target.parent.mkdir(parents=True, exist_ok=True)
    data = urllib.request.urlopen(f"{BUCKET}/{key}").read()
    target.write_bytes(data)


def main():
    keys = [k for k in list_keys() if k.endswith((".wav", ".csv"))]
    print(f"Downloading {len(keys)} files to {OUT}")
    with ThreadPoolExecutor(32) as pool:
        for i, _ in enumerate(pool.map(fetch, keys), 1):
            if i % 500 == 0:
                print(f"  {i}/{len(keys)}")
    print("Done")


if __name__ == "__main__":
    main()
```

ml/features.py

Turns a recording into a log-mel spectrogram and 4-second windows.

```
"""
Turns a heart sound recording into a log-mel spectrogram: a picture of
which frequencies are loud at each moment. The CNN looks at this picture.

web/js/features.js implements exactly the same steps in JavaScript, and
tests/parity check that both give the same numbers.
"""

import numpy as np
from scipy.io import wavfile

SAMPLE_RATE = 4000      # CirCor recordings are sampled at 4 kHz
N_FFT = 256             # 64 ms analysis frame
HOP = 64                # 16 ms between frames
N_MELS = 32
F_MIN = 25.0            # heart sounds and murmurs live roughly in 25-800 Hz
F_MAX = 1000.0
WINDOW_FRAMES = 247     # frames in one 4-second window
WINDOW_STEP = 125       # 2 seconds between window starts
EPS = 1e-6


def hz_to_mel(f):
    return 2595.0 * np.log10(1.0 + f / 700.0)


def mel_to_hz(m):
    return 700.0 * (10.0 ** (m / 2595.0) - 1.0)


def mel_filterbank():
    """Triangular filters, evenly spaced on the mel scale (how humans hear pitch)."""
    n_bins = N_FFT // 2 + 1
    bin_freqs = np.arange(n_bins) * SAMPLE_RATE / N_FFT
    mel_points = np.linspace(hz_to_mel(F_MIN), hz_to_mel(F_MAX), N_MELS + 2)
    hz_points = mel_to_hz(mel_points)
    fb = np.zeros((N_MELS, n_bins))
    for m in range(N_MELS):
        left, center, right = hz_points[m], hz_points[m + 1], hz_points[m + 2]
        rising = (bin_freqs - left) / (center - left)
        falling = (right - bin_freqs) / (right - center)
        fb[m] = np.maximum(0.0, np.minimum(rising, falling))
    return fb


MEL_FB = mel_filterbank()
HANN = 0.5 - 0.5 * np.cos(2 * np.pi * np.arange(N_FFT) / N_FFT)


def load_wav(path):
    rate, x = wavfile.read(path)
    if rate != SAMPLE_RATE:
        raise ValueError(f"{path}: expected {SAMPLE_RATE} Hz, got {rate}")
    if x.dtype == np.int16:
        x = x.astype(np.float64) / 32768.0
    return x.astype(np.float64)


def normalize(x):
    """Remove the DC offset and scale to unit loudness, so a quiet and a loud
    stethoscope give the same spectrogram."""
    x = x - x.mean()
    return x / (x.std() + 1e-8)


def log_mel(x):
    """Full-recording log-mel spectrogram, shape (N_MELS, n_frames)."""
    x = normalize(x)
    if len(x) < N_FFT:
        x = np.pad(x, (0, N_FFT - len(x)))
    n_frames = 1 + (len(x) - N_FFT) // HOP
    idx = np.arange(N_FFT)[None, :] + HOP * np.arange(n_frames)[:, None]
    frames = x[idx] * HANN
    power = np.abs(np.fft.rfft(frames, axis=1)) ** 2
    return np.log10(MEL_FB @ power.T + EPS).astype(np.float32)


def window_starts(n_frames):
    """Start frame of every 4-second window, 2 seconds apart."""
    if n_frames <= WINDOW_FRAMES:
        return [0]
    return list(range(0, n_frames - WINDOW_FRAMES + 1, WINDOW_STEP))


def cut_window(spec, start):
    """One 4-second window. Short recordings are repeated to fill it."""
    if spec.shape[1] < WINDOW_FRAMES:
```

```
        reps = int(np.ceil(WINDOW_FRAMES / spec.shape[1]))
        spec = np.tile(spec, (1, reps))
    return spec[:, start:start + WINDOW_FRAMES]
```

ml/train.py

Cross-validated training of the CNN and baseline, ONNX export, demo cases.

```
"""
LittleHeart - training and evaluation.

1. Reads the CirCor DigiScope dataset (942 children, 3,163 heart sound recordings).
2. Holds out a few children as demo cases that the model never trains on.
3. Runs 5-fold cross-validation, split by CHILD (never by recording), comparing
   our CNN against a simple baseline using the official PhysioNet 2022 score.
4. Trains the final model on all remaining children and exports it to ONNX
   for the browser, plus the metrics and demo cases.

Run:  python ml/train.py            (about 30-60 minutes on a laptop CPU)
      python ml/train.py --quick    (2 folds, few epochs, to test the pipeline)
"""

import argparse
import csv
import json
import shutil
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import StratifiedKFold, train_test_split

import features as F

ROOT = Path(__file__).resolve().parent.parent
DATA_DIR = next((ROOT / "data").glob("*circor*1.0.3*"), ROOT / "data" / "circor")
CACHE = ROOT / "data" / "spectrograms.npz"
WEB_MODEL = ROOT / "web" / "model"
WEB_SAMPLES = ROOT / "web" / "samples"

CLASSES = ["Absent", "Present", "Unknown"]
ABSENT, PRESENT, UNKNOWN = 0, 1, 2
SEED = 7


# ---------------------------------------------------------------- data

def load_patients():
    rows = list(csv.DictReader(open(DATA_DIR / "training_data.csv", encoding="utf-8")))
    patients = []
    for r in rows:
        pid = r["Patient ID"]
        murmur_locs = set(r["Murmur locations"].split("+")) if r["Murmur locations"] not in ("", "nan") else set()
        recs = []
        for wav in sorted((DATA_DIR / "training_data").glob(f"{pid}_*.wav")):
            loc = wav.stem.split("_")[1]
            if r["Murmur"] == "Present":
                # A murmur is only audible at some valves; other sites are labelled absent.
                label = PRESENT if loc in murmur_locs else ABSENT
            elif r["Murmur"] == "Unknown":
                label = UNKNOWN
            else:
                label = ABSENT
            recs.append({"path": wav, "location": loc, "label": label})
        patients.append({
            "id": pid,
            "label": CLASSES.index(r["Murmur"]),
            "age": r["Age"],
            "sex": r["Sex"],
            "murmur_locations": sorted(murmur_locs),
            "timing": r["Systolic murmur timing"],
            "grade": r["Systolic murmur grading"],
            "outcome": r["Outcome"],
            "recordings": recs,
        })
    return patients


def load_spectrograms(patients):
    if CACHE.exists():
        cached = np.load(CACHE, allow_pickle=True)
        specs = cached["specs"].item()
    else:
        specs = {}
    missing = [rec for p in patients for rec in p["recordings"] if rec["path"].name not in specs]
    if missing:
        print(f"Computing {len(missing)} spectrograms...")
        for rec in missing:
            specs[rec["path"].name] = F.log_mel(F.load_wav(rec["path"]))
        np.savez_compressed(CACHE, specs=np.array(specs, dtype=object))
    for p in patients:
```

```
        for rec in p["recordings"]:
            rec["spec"] = specs[rec["path"].name]


# ---------------------------------------------------------------- model

class MurmurCNN(nn.Module):
    """Small convolutional network: four conv blocks, then average over time
    and frequency, then 3 outputs (Absent / Present / Unknown)."""

    def __init__(self, mean, std):
        super().__init__()
        self.register_buffer("mean", torch.tensor(mean, dtype=torch.float32).view(1, 1, -1, 1))
        self.register_buffer("std", torch.tensor(std, dtype=torch.float32).view(1, 1, -1, 1))

        def block(c_in, c_out, pool=True):
            layers = [nn.Conv2d(c_in, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU()]
            if pool:
                layers.append(nn.MaxPool2d(2))
            return layers

        self.body = nn.Sequential(*block(1, 16), *block(16, 32), *block(32, 64), *block(64, 64, pool=False))
        self.head = nn.Sequential(nn.Dropout(0.3), nn.Linear(64, 3))

    def forward(self, x):                  # x: (batch, 1, N_MELS, WINDOW_FRAMES) log-mel
        x = (x - self.mean) / self.std     # standardise each mel band
        x = self.body(x)
        x = x.mean(dim=(2, 3))             # global average pooling
        return self.head(x)


class WithSoftmax(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model

    def forward(self, x):
        return torch.softmax(self.model(x), dim=1)


def band_stats(recordings):
    allspec = np.concatenate([r["spec"] for r in recordings], axis=1)
    return allspec.mean(axis=1), allspec.std(axis=1) + 1e-6


def random_window(spec, rng):
    n = spec.shape[1]
    start = 0 if n <= F.WINDOW_FRAMES else int(rng.integers(0, n - F.WINDOW_FRAMES + 1))
    return F.cut_window(spec, start)


def train_cnn(train_recs, val_patients, epochs, rng, log=print):
    torch.manual_seed(int(rng.integers(1e9)))
    mean, std = band_stats(train_recs)
    model = MurmurCNN(mean, std)
    opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-3)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    # Murmurs are rarer and matter more, so their mistakes cost more.
    loss_fn = nn.CrossEntropyLoss(weight=torch.tensor([1.0, 3.0, 2.0]))
    labels = np.array([r["label"] for r in train_recs])

    best = (-1.0, None, None)
    for epoch in range(epochs):
        model.train()
        order = np.concatenate([rng.permutation(len(train_recs)) for _ in range(3)])
        total = 0.0
        for i in range(0, len(order), 64):
            batch = order[i:i + 64]
            x = np.stack([random_window(train_recs[j]["spec"], rng) for j in batch])[:, None]
            x = x + rng.normal(0, 0.05, x.shape).astype(np.float32)
            y = torch.tensor(labels[batch])
            opt.zero_grad()
            loss = loss_fn(model(torch.from_numpy(x)), y)
            loss.backward()
            opt.step()
            total += loss.item() * len(batch)
        sched.step()

        probs = patient_probs(lambda w: predict_windows(model, w), val_patients)
        t, score = tune_thresholds(probs, [p["label"] for p in val_patients])
        log(f"    epoch {epoch + 1:2d}  loss {total / len(order):.3f}  val weighted acc {score:.3f}")
        if score > best[0]:
            best = (score, {k: v.clone() for k, v in model.state_dict().items()}, t)
    model.load_state_dict(best[1])
    model.eval()
    return model, best[2]


@torch.no_grad()
def predict_windows(model, windows):
    model.eval()
```

```
    x = torch.from_numpy(np.stack(windows)[:, None].astype(np.float32))
    return torch.softmax(model(x), dim=1).numpy()


def recording_probs(predict, spec):
    """Average the class probabilities of every 4-second window."""
    windows = [F.cut_window(spec, s) for s in F.window_starts(spec.shape[1])]
    return predict(windows).mean(axis=0)


def patient_probs(predict, patients):
    """For each child: the highest Present and Unknown probability over all valves."""
    out = []
    for p in patients:
        rec = np.array([recording_probs(predict, r["spec"]) for r in p["recordings"]])
        out.append((rec[:, PRESENT].max(), rec[:, UNKNOWN].max()))
    return np.array(out)


# ---------------------------------------------------------------- decisions and metrics

def decide(probs, t_present, t_unknown):
    """Present if any valve looks like a murmur, else Unknown if any recording
    is unclear, else Absent. Same rule order as the PhysioNet challenge."""
    out = np.full(len(probs), ABSENT)
    out[probs[:, 1] >= t_unknown] = UNKNOWN
    out[probs[:, 0] >= t_present] = PRESENT
    return out


def weighted_accuracy(y_true, y_pred):
    """Official PhysioNet 2022 murmur score: Present counts 5x, Unknown 3x, Absent 1x."""
    w = {PRESENT: 5, UNKNOWN: 3, ABSENT: 1}
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    num = sum(w[c] * np.sum((y_true == c) & (y_pred == c)) for c in w)
    den = sum(w[c] * np.sum(y_true == c) for c in w)
    return float(num / den)


def tune_thresholds(probs, y_true):
    grid = np.round(np.arange(0.05, 0.96, 0.05), 2)
    best = (-1.0, (0.5, 0.5))
    for tp in grid:
        for tu in grid:
            s = weighted_accuracy(y_true, decide(probs, tp, tu))
            if s > best[0]:
                best = (s, (float(tp), float(tu)))
    return best[1], best[0]


def screening_metrics(y_true, y_pred, present_prob):
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    known = y_true != UNKNOWN
    is_murmur = y_true[known] == PRESENT
    flagged = y_pred[known] == PRESENT
    confusion = [[int(np.sum((y_true == t) & (y_pred == p))) for p in range(3)] for t in range(3)]
    return {
        "weighted_accuracy": weighted_accuracy(y_true, y_pred),
        "sensitivity": float(np.mean(flagged[is_murmur])),
        "specificity": float(np.mean(~flagged[~is_murmur])),
        "auc": float(roc_auc_score(is_murmur, present_prob[known])),
        "confusion": confusion,  # rows = true, cols = predicted (Absent, Present, Unknown)
    }


# ---------------------------------------------------------------- baseline

def baseline_features(spec):
    return np.concatenate([spec.mean(axis=1), spec.std(axis=1)])


def train_baseline(train_recs):
    X = np.stack([baseline_features(r["spec"]) for r in train_recs])
    y = np.array([r["label"] for r in train_recs])
    clf = LogisticRegression(max_iter=3000, C=0.5, class_weight="balanced")
    mean, std = X.mean(0), X.std(0) + 1e-6
    clf.fit((X - mean) / std, y)
    return lambda spec: clf.predict_proba(((baseline_features(spec) - mean) / std)[None])[0]


def baseline_patient_probs(predict_rec, patients):
    out = []
    for p in patients:
        rec = np.array([predict_rec(r["spec"]) for r in p["recordings"]])
        out.append((rec[:, PRESENT].max(), rec[:, UNKNOWN].max()))
    return np.array(out)


# ---------------------------------------------------------------- main

def pick_demo_patients(patients, rng):
```

```
    """A few children per class that are never used for training or CV."""
    chosen = []
    for label, n in [(PRESENT, 4), (ABSENT, 4), (UNKNOWN, 2)]:
        pool = [p for p in patients if p["label"] == label and 3 <= len(p["recordings"]) <= 5]
        chosen += list(rng.choice(pool, n, replace=False))
    return chosen


def summarize(fold_metrics):
    keys = ["weighted_accuracy", "sensitivity", "specificity", "auc"]
    return {k: {"mean": float(np.mean([m[k] for m in fold_metrics])),
                "std": float(np.std([m[k] for m in fold_metrics]))} for k in keys}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true")
    args = ap.parse_args()
    n_folds, epochs = (2, 2) if args.quick else (5, 20)
    rng = np.random.default_rng(SEED)
    t0 = time.time()

    patients = load_patients()
    load_spectrograms(patients)
    demo = pick_demo_patients(patients, rng)
    demo_ids = {p["id"] for p in demo}
    pool = [p for p in patients if p["id"] not in demo_ids]
    n_recs = sum(len(p["recordings"]) for p in patients)
    print(f"{len(patients)} children, {n_recs} recordings; {len(demo)} held out as demo cases")

    y_pool = np.array([p["label"] for p in pool])
    folds = StratifiedKFold(n_folds, shuffle=True, random_state=SEED).split(np.zeros(len(pool)), y_pool)
    cnn_folds, base_folds = [], []
    oof_true, oof_pred = [], []

    for k, (tr_idx, te_idx) in enumerate(folds):
        train_p = [pool[i] for i in tr_idx]
        test_p = [pool[i] for i in te_idx]
        inner_train, inner_val = train_test_split(
            train_p, test_size=0.15, random_state=SEED, stratify=[p["label"] for p in train_p])
        train_recs = [r for p in inner_train for r in p["recordings"]]
        y_test = [p["label"] for p in test_p]
        print(f"Fold {k + 1}/{n_folds}: train {len(inner_train)}, val {len(inner_val)}, test {len(test_p)}
children")

        model, (tp, tu) = train_cnn(train_recs, inner_val, epochs, rng)
        probs = patient_probs(lambda w: predict_windows(model, w), test_p)
        pred = decide(probs, tp, tu)
        m = screening_metrics(y_test, pred, probs[:, 0])
        cnn_folds.append(m)
        oof_true += y_test
        oof_pred += list(pred)

        base = train_baseline(train_recs)
        bt, _ = tune_thresholds(baseline_patient_probs(base, inner_val), [p["label"] for p in inner_val])
        bprobs = baseline_patient_probs(base, test_p)
        base_folds.append(screening_metrics(y_test, decide(bprobs, *bt), bprobs[:, 0]))
        print(f"  CNN weighted acc {m['weighted_accuracy']:.3f} (sens {m['sensitivity']:.2f}, "
              f"spec {m['specificity']:.2f}, AUC {m['auc']:.3f})  |  baseline {base_folds[-1][
'weighted_accuracy']:.3f}")

    cnn_summary, base_summary = summarize(cnn_folds), summarize(base_folds)
    confusion = [[int(np.sum((np.array(oof_true) == t) & (np.array(oof_pred) == p))) for p in range(3)] for t in
range(3)]
    print("CNN     ", {k: round(v["mean"], 3) for k, v in cnn_summary.items()})
    print("Baseline", {k: round(v["mean"], 3) for k, v in base_summary.items()})

    print("Training final model on all non-demo children...")
    final_train, final_val = train_test_split(pool, test_size=0.15, random_state=SEED, stratify=y_pool)
    model, (tp, tu) = train_cnn([r for p in final_train for r in p["recordings"]], final_val, epochs, rng)
    export(model, tp, tu, cnn_summary, base_summary, confusion, n_folds, len(pool), demo, len(patients), n_recs)
    print(f"Done in {(time.time() - t0) / 60:.1f} min")


def export(model, tp, tu, cnn_summary, base_summary, confusion, n_folds, n_pool, demo, n_patients, n_recs):
    WEB_MODEL.mkdir(parents=True, exist_ok=True)
    WEB_SAMPLES.mkdir(parents=True, exist_ok=True)
    dummy = torch.zeros(1, 1, F.N_MELS, F.WINDOW_FRAMES)
    torch.onnx.export(WithSoftmax(model).eval(), dummy, str(WEB_MODEL / "murmur_cnn.onnx"),
                      input_names=["logmel"], output_names=["probs"],
                      dynamic_axes={"logmel": {0: "batch"}, "probs": {0: "batch"}},
                      opset_version=17, dynamo=False)

    config = {
        "features": {
            "sample_rate": F.SAMPLE_RATE, "n_fft": F.N_FFT, "hop": F.HOP, "n_mels": F.N_MELS,
            "window_frames": F.WINDOW_FRAMES, "window_step": F.WINDOW_STEP, "eps": F.EPS,
            "mel_filterbank": np.round(F.MEL_FB, 7).tolist(),
        },
        "classes": CLASSES,
        "thresholds": {"present": tp, "unknown": tu},
```

```
    }
    (WEB_MODEL / "config.json").write_text(json.dumps(config))

    demo_cases = []
    for p in demo:
        files = []
        for r in p["recordings"]:
            shutil.copy(r["path"], WEB_SAMPLES / r["path"].name)
            files.append({"file": r["path"].name, "location": r["location"]})
        demo_cases.append({"id": p["id"], "truth": CLASSES[p["label"]], "age": p["age"], "sex": p["sex"],
                           "murmur_locations": p["murmur_locations"], "timing": p["timing"],
                           "grade": p["grade"], "recordings": files})

    metrics = {
        "dataset": {"children": n_patients, "recordings": n_recs, "cv_children": n_pool, "demo_children": len(
demo)},
        "folds": n_folds,
        "cnn": cnn_summary,
        "baseline": base_summary,
        "confusion": confusion,
        "challenge_reference": {"top_hidden_test": 0.780, "note": "Best team, PhysioNet 2022 hidden test set"},
        "demo_cases": demo_cases,
    }
    (WEB_MODEL / "metrics.json").write_text(json.dumps(metrics, indent=1))
    (WEB_MODEL / "metrics.js").write_text("const METRICS = " + json.dumps(metrics) + ";\n")
    print(f"Exported model, thresholds ({tp}, {tu}) and {len(demo_cases)} demo cases to web/")


if __name__ == "__main__":
    main()
```

ml/heartbeat_check.py

Safety check calibration: refuse recordings without a heartbeat rhythm.

```
"""
Safety check: does a recording contain a heartbeat rhythm at all?

The CNN was only trained on heart sounds, so it can give confident but
meaningless answers for speech, noise or a phone lying on a table. Before
trusting the CNN, we look for a repeating beat between 40 and 220 beats per
minute in the loudness of the low frequencies (where S1 and S2 live).

The threshold is calibrated so that 98% of real CirCor recordings pass,
then tested on signals that are not heart sounds.

Run:  python ml/heartbeat_check.py      (after ml/train.py has cached spectrograms)
"""

import json
from pathlib import Path

import numpy as np

import features as F

ROOT = Path(__file__).resolve().parent.parent
FRAME_RATE = F.SAMPLE_RATE / F.HOP           # 62.5 spectrogram frames per second
LOW_BANDS = 12                               # mel bands up to roughly 200 Hz
MIN_LAG = int(FRAME_RATE * 60 / 220)         # 220 bpm
MAX_LAG = int(FRAME_RATE * 60 / 40)          # 40 bpm


def beat_measures(spec):
    """Returns (rhythm, depth) of the low-frequency loudness curve.
    rhythm: highest autocorrelation at a heartbeat-like delay (0 = none, 1 = perfectly regular).
    depth:  how much the loudness rises and falls (standard deviation, in log10 units);
            heartbeats are loud clicks with quiet gaps, a steady tone is flat."""
    env = spec[:LOW_BANDS].mean(axis=0).astype(np.float64)
    env = env - env.mean()
    denom = np.dot(env, env)
    depth = float(np.sqrt(denom / max(1, len(env))))
    if denom <= 0 or len(env) <= MAX_LAG + 1:
        return 0.0, depth
    best = 0.0
    for lag in range(MIN_LAG, MAX_LAG + 1):
        best = max(best, np.dot(env[:-lag], env[lag:]) / denom)
    return float(best), depth


def synthetic_negatives(rng, seconds=15):
    t = np.arange(F.SAMPLE_RATE * seconds) / F.SAMPLE_RATE
    n = len(t)
    syllables = np.zeros(n)
    pos = 0
    while pos < n:  # irregular syllables (0.08-0.35 s) separated by irregular gaps
        length = int(rng.uniform(0.08, 0.35) * F.SAMPLE_RATE)
        syllables[pos:pos + length] = rng.uniform(0.5, 1.0)
        pos += length + int(rng.uniform(0.03, 0.4) * F.SAMPLE_RATE)
    pitch = 120 + 30 * np.sin(2 * np.pi * 0.3 * t)
    return {
        "white noise": rng.normal(size=n),
        "pink-ish noise": np.cumsum(rng.normal(size=n)) * 0.01 + rng.normal(size=n),
        "steady tone 440 Hz": np.sin(2 * np.pi * 440 * t),
        "mains hum 50 Hz": np.sin(2 * np.pi * 50 * t) + 0.1 * rng.normal(size=n),
        "speech-like babble": syllables * np.sin(2 * np.pi * np.cumsum(pitch) / F.SAMPLE_RATE) + 0.05 * rng.
normal(size=n),
        "silence with hiss": 0.001 * rng.normal(size=n),
    }


def main():
    specs = np.load(ROOT / "data" / "spectrograms.npz", allow_pickle=True)["specs"].item()
    real = np.array([beat_measures(s) for s in specs.values()])
    # The rhythm limit lets 97% of real recordings through, the depth limit 99%.
    t_rhythm = float(np.round(np.percentile(real[:, 0], 3), 3))
    t_depth = float(np.round(np.percentile(real[:, 1], 1), 3))
    passes = lambda m: m[0] >= t_rhythm and m[1] >= t_depth
    real_pass = float(np.mean([passes(m) for m in real]))
    print(f"{len(real)} real recordings: median rhythm {np.median(real[:, 0]):.3f}, depth {np.median(real[:, 1]):.
3f}")
    print(f"limits: rhythm >= {t_rhythm}, depth >= {t_depth}; real recordings passing: {real_pass:.1%}")

    negatives = {}
    trials = [synthetic_negatives(np.random.default_rng(seed)) for seed in range(10)]
    for name in trials[0]:
        rejected = sum(not passes(beat_measures(F.log_mel(t[name]))) for t in trials)
        negatives[name] = {"rejected": rejected, "trials": len(trials)}
        print(f"  {name:20s} rejected {rejected}/{len(trials)}")

    out = {
        "rhythm_min": t_rhythm,
```

```
        "depth_min": t_depth,
        "low_bands": LOW_BANDS,
        "min_lag": MIN_LAG,
        "max_lag": MAX_LAG,
        "real_pass_rate": real_pass,
        "real_recordings": int(len(real)),
        "negatives": negatives,
    }
    (ROOT / "web" / "model" / "heartbeat_check.json").write_text(json.dumps(out, indent=1))
    print("Saved web/model/heartbeat_check.json")


if __name__ == "__main__":
    main()
```

web/index.html

Page structure: screening, validation and education tabs.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>LittleHeart</title>
  <meta name="description" content="AI heart murmur screening for children, running privately in the browser.">
  <link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 32 32'><path d='M16 28s-11-7-11-15a6 6 0 0 1 11-3 6 6 0 0 1 11 3c0 8-11 15-11 15z' fill='%23d6455d'/></svg>">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="wrap header-row">
      <div class="brand">
        <svg viewBox="0 0 32 32" class="logo" aria-hidden="true"><path d="M16 28s-11-7-11-15a6 6 0 0 1 11-3 6 6 0 0 1 11 3c0 8-11 15-11 15z"/><path d="M6 16h6l2-4 3 8 2-4h7" class="logo-line"/></svg>
        <span>LittleHeart</span>
      </div>
      <nav class="tabs" role="tablist">
        <button class="tab active" data-tab="screen" role="tab">Screen a child</button>
        <button class="tab" data-tab="validation" role="tab">Validation</button>
        <button class="tab" data-tab="learn" role="tab">About murmurs</button>
      </nav>
    </div>
  </header>

  <main class="wrap">
    <!-- ==================== SCREEN ==================== -->
    <section id="tab-screen" class="panel active">
      <div class="intro">
        <h1>Hear the heart problems that hide in children.</h1>
        <p>LittleHeart listens to stethoscope recordings from the four heart valve areas and flags children whose heart
          sounds contain a murmur, so a health worker knows who needs an echocardiogram. The AI runs on this device; no recording is uploaded.</p>
      </div>

      <div class="screen-grid">
        <div class="card input-card">
          <h2>1. Add recordings</h2>
          <div class="source-tabs" role="tablist">
            <button class="source active" data-source="demo">Real cases</button>
            <button class="source" data-source="upload">Upload</button>
            <button class="source" data-source="record">Record</button>
          </div>

          <div class="source-panel active" id="source-demo">
            <p class="muted small">Children from the CirCor dataset that the AI never saw during training. The cardiologist's finding is revealed after the analysis.</p>
            <div id="demo-cases" class="demo-cases"></div>
          </div>

          <div class="source-panel" id="source-upload">
            <p class="muted small">Choose WAV/MP3 recordings. Name files like <code>name_AV.wav</code>, <code>name_MV.wav</code> to set the valve automatically.</p>
            <label class="file-drop" id="file-drop">
              <input type="file" id="file-input" accept="audio/*,.wav" multiple>
              <span><b>Choose files</b> or drop them here</span>
            </label>
          </div>

          <div class="source-panel" id="source-record">
            <p class="muted small">Place a digital stethoscope, or a stethoscope with a phone adapter, on the highlighted spot. Record 15 seconds while the child is calm.</p>
            <div class="record-row">
              <select id="record-site" aria-label="Valve area">
                <option value="AV">Aortic (AV)</option>
                <option value="PV">Pulmonary (PV)</option>
                <option value="TV">Tricuspid (TV)</option>
                <option value="MV">Mitral (MV)</option>
              </select>
              <button class="btn" id="record-btn">● Record 15 s</button>
            </div>
            <p id="record-status" class="small muted" aria-live="polite"></p>
          </div>

          <h2 class="mt">2. Listening sites</h2>
          <div class="chest-wrap">
            <svg viewBox="30 20 160 170" class="chest" id="chest" aria-label="Chest diagram with the four auscultation sites">
              <path class="torso" d="M40 20 Q110 0 180 20 L195 70 Q200 150 185 220 L35 220 Q20 150 25 70 Z"/>
              <path class="sternum" d="M110 30 L110 150"/>
              <path class="heart-shape" d="M95 85 Q80 70 95 62 Q110 58 112 75 Q125 60 140 70 Q150 85 135 110 L112 135 Q95 115 95 85Z"/>
              <g class="site" data-site="AV" transform="translate(92 62)"><circle r="13"/><text>AV</text></g>
              <g class="site" data-site="PV" transform="translate(128 62)"><circle r="13"/><text>PV</text></g>
```

```
              <g class="site" data-site="TV" transform="translate(118 118)"><circle r="13"/><text>TV</text></g>
              <g class="site" data-site="MV" transform="translate(150 132)"><circle r="13"/><text>MV</text></g>
            </svg>
            <ul class="site-legend">
              <li><b>AV</b> aortic · 2nd space, right of sternum</li>
              <li><b>PV</b> pulmonary · 2nd space, left of sternum</li>
              <li><b>TV</b> tricuspid · lower left sternum</li>
              <li><b>MV</b> mitral · apex, 5th space</li>
            </ul>
          </div>
        </div>

        <div class="results-col">
          <div class="card verdict-card" id="verdict-card">
            <h2>3. Result</h2>
            <div id="verdict" class="verdict empty">
              <div class="verdict-icon" aria-hidden="true">■</div>
              <div>
                <div class="verdict-title">No recordings yet</div>
                <div class="verdict-text">Pick a real case or add recordings to start.</div>
              </div>
            </div>
            <div id="truth" class="truth" hidden></div>
            <div class="verdict-actions" id="verdict-actions" hidden>
              <button class="btn-ghost" id="print-btn">Print screening note</button>
              <button class="btn-ghost" id="clear-btn">New child</button>
            </div>
            <p class="print-only small" id="print-meta"></p>
          </div>

          <div id="site-results" class="site-results"></div>
        </div>
      </div>

      <p class="disclaimer">LittleHeart is a research prototype for screening support, not a medical device and not a diagnosis.
        It was trained on children in north-east Brazil recorded with one stethoscope model; accuracy on other devices and populations is unknown.
        A child who seems unwell should see a clinician whatever the result.</p>
    </section>

    <!-- ==================== VALIDATION ==================== -->
    <section id="tab-validation" class="panel">
      <div class="intro">
        <h1>How well does it work?</h1>
        <p id="val-intro"></p>
      </div>

      <div class="grid four" id="val-cards"></div>

      <div class="grid two">
        <div class="card">
          <h2>Compared with other approaches</h2>
          <p class="muted small">Official PhysioNet 2022 murmur score (weighted accuracy). Higher is better.</p>
          <div id="compare" class="compare"></div>
          <p class="muted small" id="compare-note"></p>
        </div>
        <div class="card">
          <h2>Every child in cross-validation</h2>
          <p class="muted small">Rows: cardiologist's finding. Columns: LittleHeart's answer.</p>
          <div id="confusion" class="confusion"></div>
        </div>
      </div>

      <div class="grid two">
        <div class="card">
          <h2>How the AI works</h2>
          <ol class="steps">
            <li><b>Spectrogram.</b> Each recording is turned into a log-mel spectrogram: a picture of which pitches (25–1000 Hz) are loud every 16 ms.</li>
            <li><b>4-second windows.</b> The picture is cut into overlapping 4-second windows, about 4–8 heartbeats each.</li>
            <li><b>CNN.</b> A small convolutional neural network (4 layers, about 61,000 weights) scores each window: murmur, no murmur, or unclear.</li>
            <li><b>Per valve.</b> Window scores are averaged for each recording.</li>
            <li><b>Per child.</b> If any valve looks like a murmur, the child is referred. If none does but a recording is unclear, it asks for a re-recording.</li>
          </ol>
        </div>
        <div class="card">
          <h2>How we tested it</h2>
          <ul class="checks">
            <li><b>Split by child, not recording.</b> All recordings of a child are either in training or in testing, never both.</li>
            <li><b>Cross-validation.</b> Every child is tested once by a model that never heard them.</li>
            <li><b>Thresholds chosen on separate data.</b> Decision cut-offs come from a validation split inside the training folds.</li>
            <li><b>Baseline.</b> A logistic regression on average spectrum shows how much the CNN adds.</li>
            <li><b>Untouched demo cases.</b> The 10 real cases on the first tab were picked at random and removed before any training.
              LittleHeart agrees with the cardiologist on 7. The other 3 are children without a murmur that it
```

```
asks to re-record; it misses none of the 4 murmurs.</li>
          </ul>
        </div>
      </div>

      <div class="card safety-card">
        <h2>Safety check: is it really a heartbeat?</h2>
        <p class="muted small">The CNN has only ever heard heart sounds, so it could give a confident but meaningless answer for speech or noise.
          Before trusting it, LittleHeart checks that the low-pitched loudness rises and falls with a regular rhythm between 40 and 220 beats per minute.
          Recordings that fail are never called "murmur" or "no murmur"; the health worker is asked to record again.</p>
        <div id="safety" class="safety"></div>
      </div>

      <div class="card">
        <h2>Honest status</h2>
        <div class="status-grid">
          <div><h3 class="ok-text">Working now</h3><ul id="status-done">
            <li>Trained, cross-validated murmur model on 942 children</li>
            <li>In-browser analysis of real recordings, uploads and microphone input</li>
            <li>Per-valve explanation: spectrogram with murmur timeline</li>
            <li>Python and JavaScript feature code verified to match</li>
            <li>Heartbeat safety check that refuses noise, tones and most speech</li>
          </ul></div>
          <div><h3 class="warn-text">Not done yet</h3><ul>
            <li>Testing with other stethoscopes and phone adapters</li>
            <li>Validation in a new population or prospective study</li>
            <li>Telling innocent from pathological murmurs</li>
            <li>Clinical review and regulatory approval</li>
          </ul></div>
          <div><h3 class="bad-text">Known limits</h3><ul>
            <li>Misses about 1 in <span id="miss-rate">4</span> murmurs in testing</li>
            <li>Trained on one region (Brazil) and one device</li>
            <li>Cross-validation on public data is not the challenge's hidden test set</li>
            <li>Plain phone microphones are much noisier than stethoscopes</li>
            <li>Rhythmic sounds that are not heartbeats (e.g. a metronome) can pass the safety check</li>
            <li>Validation scores above do not include the heartbeat check, which asks for a re-recording of about 4% of real recordings</li>
          </ul></div>
        </div>
      </div>
    </section>

    <!-- ================== LEARN ================== -->
    <section id="tab-learn" class="panel">
      <div class="intro">
        <h1>Why listen to children's hearts?</h1>
        <p>Congenital heart disease is the most common birth defect, and rheumatic heart disease still damages the hearts of millions of
          children and young people. Both are often first noticed as a murmur, but echocardiography and cardiologists are scarce where these diseases are most common.</p>
      </div>
      <div class="grid three">
        <article class="card"><h2>What is a murmur?</h2>
          <p>A whooshing sound between the normal "lub-dub" heart sounds, caused by turbulent blood flow. Many murmurs in children are innocent, but some come from holes in the heart or damaged valves.</p></article>
        <article class="card"><h2>Who is LittleHeart for?</h2>
          <p>Community health workers, school screening programmes and primary care clinics that have a stethoscope but no cardiologist, and need to decide which children to send for an echocardiogram.</p></article>
        <article class="card"><h2>Why AI?</h2>
          <p>Hearing a murmur takes years of practice, and even trained listeners disagree. An AI that has heard thousands of recordings can give every health worker a consistent second opinion.</p></article>
        <article class="card"><h2>The four sites</h2>
          <p>Each valve is best heard at one spot: aortic and pulmonary in the 2nd rib space either side of the breastbone, tricuspid at its lower left edge, and mitral at the apex of the heart.</p></article>
        <article class="card"><h2>Recording tips</h2>
          <p>Use a quiet room, keep the child calm and still, press the stethoscope firmly on bare skin, and record at least 10 seconds per site. Crying and talking make recordings unclear.</p></article>
        <article class="card"><h2>What happens next?</h2>
          <p>A "refer" result means the child should have an echocardiogram, the ultrasound test that shows the heart's structure. A "no murmur" result does not rule out heart disease.</p></article>
      </div>
      <p class="muted small">Data: CirCor DigiScope Phonocardiogram Dataset (Oliveira et al., IEEE JBHI 2022), PhysioNet, CC BY 4.0.</p>
    </section>
  </main>

  <footer class="wrap footer">LittleHeart · UnivaBio 2026: AI for Human Health · Research prototype, not a medical device</footer>

  <script src="https://cdn.jsdelivr.net/npm/onnxruntime-web@1.20.1/dist/ort.min.js"></script>
  <script src="model/metrics.js"></script>
  <script src="js/features.js"></script>
  <script src="js/model.js"></script>
  <script src="js/app.js"></script>
</body>
</html>
```

web/js/features.js

JavaScript port of features.py, so the browser computes identical spectrograms.

```
// Audio loading and log-mel spectrogram, a line-by-line port of ml/features.py.
// tests/parity.test.mjs checks that both versions produce the same numbers.

(function (root) {
  "use strict";

  // Reads a 16-bit PCM WAV file (the CirCor format). Returns null for anything else,
  // so the caller can fall back to the browser's decoder.
  function parseWav(buffer) {
    const view = new DataView(buffer);
    const tag = (o) => String.fromCharCode(view.getUint8(o), view.getUint8(o + 1), view.getUint8(o + 2), view.
getUint8(o + 3));
    if (buffer.byteLength < 44 || tag(0) !== "RIFF" || tag(8) !== "WAVE") return null;
    let offset = 12, fmt = null;
    while (offset + 8 <= buffer.byteLength) {
      const id = tag(offset), size = view.getUint32(offset + 4, true);
      if (id === "fmt ") {
        fmt = {
          format: view.getUint16(offset + 8, true),
          channels: view.getUint16(offset + 10, true),
          sampleRate: view.getUint32(offset + 12, true),
          bits: view.getUint16(offset + 22, true),
        };
      } else if (id === "data" && fmt) {
        if (fmt.format !== 1 || fmt.bits !== 16) return null;
        const n = Math.floor(Math.min(size, buffer.byteLength - offset - 8) / (2 * fmt.channels));
        const samples = new Float64Array(n);
        for (let i = 0; i < n; i++) {
          let sum = 0;
          for (let c = 0; c < fmt.channels; c++) {
            sum += view.getInt16(offset + 8 + 2 * (i * fmt.channels + c), true) / 32768;
          }
          samples[i] = sum / fmt.channels;
        }
        return { sampleRate: fmt.sampleRate, samples };
      }
      offset += 8 + size + (size % 2);
    }
    return null;
  }

  // Converts any audio the browser can decode (mic recording, mp3, 44.1 kHz wav...)
  // to mono at the model's sample rate.
  async function decodeToRate(buffer, sampleRate) {
    const wav = parseWav(buffer.slice(0));
    if (wav && wav.sampleRate === sampleRate) return wav.samples;
    const decoded = await new OfflineAudioContext(1, 1, 44100).decodeAudioData(buffer.slice(0));
    try {
      return await resample(decoded, sampleRate);
    } catch {
      // Some browsers refuse very low sample rates: render at 2x, then average sample pairs.
      const double = await resample(decoded, sampleRate * 2);
      const out = new Float64Array(Math.floor(double.length / 2));
      for (let i = 0; i < out.length; i++) out[i] = (double[2 * i] + double[2 * i + 1]) / 2;
      return out;
    }
  }

  async function resample(decoded, sampleRate) {
    const length = Math.ceil(decoded.duration * sampleRate);
    const offline = new OfflineAudioContext(1, length, sampleRate);
    const src = offline.createBufferSource();
    src.buffer = decoded;
    src.connect(offline.destination);
    src.start();
    const rendered = await offline.startRendering();
    return Float64Array.from(rendered.getChannelData(0));
  }

  function normalize(x) {
    let mean = 0;
    for (let i = 0; i < x.length; i++) mean += x[i];
    mean /= x.length;
    let variance = 0;
    for (let i = 0; i < x.length; i++) variance += (x[i] - mean) ** 2;
    const std = Math.sqrt(variance / x.length) + 1e-8;
    const out = new Float64Array(x.length);
    for (let i = 0; i < x.length; i++) out[i] = (x[i] - mean) / std;
    return out;
  }

  // In-place iterative radix-2 FFT on separate real/imaginary arrays.
  function fft(re, im) {
    const n = re.length;
    for (let i = 1, j = 0; i < n; i++) {
      let bit = n >> 1;
      for (; j & bit; bit >>= 1) j ^= bit;
```

```
      j ^= bit;
      if (i < j) {
        [re[i], re[j]] = [re[j], re[i]];
        [im[i], im[j]] = [im[j], im[i]];
      }
    }
    for (let len = 2; len <= n; len <<= 1) {
      const angle = (-2 * Math.PI) / len;
      for (let i = 0; i < n; i += len) {
        for (let k = 0; k < len / 2; k++) {
          const wr = Math.cos(angle * k), wi = Math.sin(angle * k);
          const ar = re[i + k + len / 2], ai = im[i + k + len / 2];
          const tr = ar * wr - ai * wi, ti = ar * wi + ai * wr;
          re[i + k + len / 2] = re[i + k] - tr;
          im[i + k + len / 2] = im[i + k] - ti;
          re[i + k] += tr;
          im[i + k] += ti;
        }
      }
    }
  }

  // Returns { data: Float32Array(nMels * nFrames) in row-major (mel, frame) order, nFrames }.
  function logMel(samples, cfg) {
    const { n_fft: nFft, hop, n_mels: nMels, eps, mel_filterbank: fb } = cfg;
    let x = normalize(samples);
    if (x.length < nFft) {
      const padded = new Float64Array(nFft);
      padded.set(x);
      x = padded;
    }
    const nFrames = 1 + Math.floor((x.length - nFft) / hop);
    const nBins = nFft / 2 + 1;
    const hann = new Float64Array(nFft);
    for (let i = 0; i < nFft; i++) hann[i] = 0.5 - 0.5 * Math.cos((2 * Math.PI * i) / nFft);

    const out = new Float32Array(nMels * nFrames);
    const re = new Float64Array(nFft), im = new Float64Array(nFft), power = new Float64Array(nBins);
    for (let f = 0; f < nFrames; f++) {
      for (let i = 0; i < nFft; i++) {
        re[i] = x[f * hop + i] * hann[i];
        im[i] = 0;
      }
      fft(re, im);
      for (let b = 0; b < nBins; b++) power[b] = re[b] * re[b] + im[b] * im[b];
      for (let m = 0; m < nMels; m++) {
        let sum = 0;
        const row = fb[m];
        for (let b = 0; b < nBins; b++) if (row[b] !== 0) sum += row[b] * power[b];
        out[m * nFrames + f] = Math.log10(sum + eps);
      }
    }
    return { data: out, nFrames };
  }

  function windowStarts(nFrames, cfg) {
    if (nFrames <= cfg.window_frames) return [0];
    const starts = [];
    for (let s = 0; s + cfg.window_frames <= nFrames; s += cfg.window_step) starts.push(s);
    return starts;
  }

  // One window, shape (nMels, windowFrames). Short recordings are repeated to fill it.
  function cutWindow(spec, start, cfg) {
    const { n_mels: nMels, window_frames: w } = cfg;
    const out = new Float32Array(nMels * w);
    for (let m = 0; m < nMels; m++) {
      for (let t = 0; t < w; t++) {
        out[m * w + t] = spec.data[m * spec.nFrames + ((start + t) % spec.nFrames)];
      }
    }
    return out;
  }

  // Port of beat_measures in ml/heartbeat_check.py: is there a heartbeat rhythm at all?
  function beatMeasures(spec, check) {
    const n = spec.nFrames;
    const env = new Float64Array(n);
    for (let f = 0; f < n; f++) {
      let sum = 0;
      for (let m = 0; m < check.low_bands; m++) sum += spec.data[m * n + f];
      env[f] = sum / check.low_bands;
    }
    let mean = 0;
    for (let f = 0; f < n; f++) mean += env[f];
    mean /= n;
    let denom = 0;
    for (let f = 0; f < n; f++) { env[f] -= mean; denom += env[f] * env[f]; }
    const depth = Math.sqrt(denom / Math.max(1, n));
    if (denom <= 0 || n <= check.max_lag + 1) return { rhythm: 0, depth };
    let rhythm = 0;
```

```
    for (let lag = check.min_lag; lag <= check.max_lag; lag++) {
      let r = 0;
      for (let f = 0; f + lag < n; f++) r += env[f] * env[f + lag];
      rhythm = Math.max(rhythm, r / denom);
    }
    return { rhythm, depth };
  }

  const api = { parseWav, decodeToRate, normalize, fft, logMel, windowStarts, cutWindow, beatMeasures };
  if (typeof module !== "undefined" && module.exports) module.exports = api;
  else root.LHFeatures = api;
})(typeof self !== "undefined" ? self : this);
```

web/js/model.js

Runs the ONNX model in the browser and applies the referral rule.

```
// Runs the exported CNN in the browser with ONNX Runtime Web and applies the
// same decision rule as ml/train.py (decide / patient_probs), plus the
// heartbeat safety check from ml/heartbeat_check.py.

(function (root) {
  "use strict";

  const ABSENT = 0, PRESENT = 1, UNKNOWN = 2;
  let session = null;
  let config = null;
  let beatCheck = null;
  let loading = null;

  function load() {
    loading = loading || (async () => {
      [config, beatCheck] = await Promise.all([
        fetch("model/config.json").then(r => r.json()),
        fetch("model/heartbeat_check.json").then(r => r.json()),
      ]);
      ort.env.wasm.wasmPaths = "https://cdn.jsdelivr.net/npm/onnxruntime-web@1.20.1/dist/";
      ort.env.wasm.numThreads = 1;
      session = await ort.InferenceSession.create("model/murmur_cnn.onnx", { executionProviders: ["wasm"] });
    })();
    return loading;
  }

  // ONNX Runtime can only run one inference at a time, so calls wait in line.
  let queue = Promise.resolve();
  function analyzeRecording(samples) {
    const job = queue.then(() => analyzeNow(samples));
    queue = job.catch(() => {});
    return job;
  }

  // Analyses one recording: spectrogram, heartbeat check, 4-second windows, CNN, average.
  async function analyzeNow(samples) {
    await load();
    const f = config.features;
    const spec = LHFeatures.logMel(samples, f);
    const m = LHFeatures.beatMeasures(spec, beatCheck);
    const beat = { ...m, ok: m.rhythm >= beatCheck.rhythm_min && m.depth >= beatCheck.depth_min };

    const starts = LHFeatures.windowStarts(spec.nFrames, f);
    const size = f.n_mels * f.window_frames;
    const batch = new Float32Array(starts.length * size);
    starts.forEach((s, i) => batch.set(LHFeatures.cutWindow(spec, s, f), i * size));
    const input = new ort.Tensor("float32", batch, [starts.length, 1, f.n_mels, f.window_frames]);
    const { probs } = await session.run({ logmel: input });
    const windows = starts.map((s, i) => ({
      startSec: (s * f.hop) / f.sample_rate,
      probs: Array.from(probs.data.slice(i * 3, i * 3 + 3)),
    }));
    const mean = [0, 1, 2].map(c => windows.reduce((sum, w) => sum + w.probs[c], 0) / windows.length);
    return { spec, windows, probs: mean, beat, duration: samples.length / f.sample_rate };
  }

  // Label for one recording. A recording without a heartbeat rhythm is never
  // trusted as "murmur" or "no murmur": it must be recorded again.
  function recordingLabel(r) {
    const t = config.thresholds;
    if (!r.beat.ok) return UNKNOWN;
    if (r.probs[PRESENT] >= t.present) return PRESENT;
    if (r.probs[UNKNOWN] >= t.unknown) return UNKNOWN;
    return ABSENT;
  }

  // Child-level decision: Present if any valve is Present, else Unknown if
  // any recording is unclear, else Absent.
  function decide(results) {
    const labels = results.map(recordingLabel);
    const trusted = results.filter(r => r.beat.ok);
    const present = trusted.length ? Math.max(...trusted.map(r => r.probs[PRESENT])) : 0;
    let label = ABSENT;
    if (labels.includes(UNKNOWN)) label = UNKNOWN;
    if (labels.includes(PRESENT)) label = PRESENT;
    return { label, labels, present, noBeat: results.filter(r => !r.beat.ok).length, thresholds: config.
thresholds };
  }

  root.LHModel = {
    load, analyzeRecording, decide, recordingLabel, ABSENT, PRESENT, UNKNOWN,
    get config() { return config; },
    get beatCheck() { return beatCheck; },
  };
})(self);
```

web/js/app.js

User interface: cases, uploads, microphone, spectrograms, results, validation charts.

```
// LittleHeart user interface.

const SITE_NAMES = { AV: "Aortic", PV: "Pulmonary", TV: "Tricuspid", MV: "Mitral", Phc: "Other site" };
const LABEL_TEXT = ["No murmur", "Murmur", "Unclear"];
const LABEL_CLASS = ["ok", "bad", "warn"];

let recordings = [];   // { id, site, name, url, result }
let currentCase = null;
let nextId = 1;

const $ = (sel) => document.querySelector(sel);
const pct = (x) => Math.round(x * 100) + "%";

// ------------------------------------------------------------ tabs

function showTab(name) {
  document.querySelectorAll(".tab").forEach(t => t.classList.toggle("active", t.dataset.tab === name));
  document.querySelectorAll(".panel").forEach(p => p.classList.toggle("active", p.id === "tab-" + name));
}

function setupTabs() {
  document.querySelectorAll(".tab").forEach(tab => tab.addEventListener("click", () => {
    showTab(tab.dataset.tab);
    history.replaceState(null, "", "#" + tab.dataset.tab);
  }));
  const fromUrl = location.hash.slice(1);
  if (document.getElementById("tab-" + fromUrl)) showTab(fromUrl);

  document.querySelectorAll(".source").forEach(btn => btn.addEventListener("click", () => {
    document.querySelectorAll(".source").forEach(b => b.classList.toggle("active", b === btn));
    document.querySelectorAll(".source-panel").forEach(p =>
      p.classList.toggle("active", p.id === "source-" + btn.dataset.source));
  }));
}

// ------------------------------------------------------------ adding recordings

// Adds all recordings first (so the result waits for every one of them),
// then analyses them one by one. Results for recordings that were cleared
// in the meantime (e.g. the user picked another case) are thrown away.
async function addRecordings(items) {
  const recs = items.map(({ buffer, site, name }) => ({
    id: nextId++, site, name, buffer, url: URL.createObjectURL(new Blob([buffer])), result: null,
  }));
  recordings.push(...recs);
  renderAll();
  for (const rec of recs) {
    if (!recordings.includes(rec)) return;
    try {
      const samples = await LHFeatures.decodeToRate(rec.buffer, LHModel.config?.features.sample_rate || 4000);
      const result = await LHModel.analyzeRecording(samples);
      if (!recordings.includes(rec)) return;
      rec.result = result;
    } catch (err) {
      console.error(err);
      rec.error = "Could not read this audio file.";
    }
    delete rec.buffer;
    renderAll();
  }
}

function renderDemoCases() {
  const cases = METRICS.demo_cases;
  $("#demo-cases").innerHTML = cases.map((c, i) => `
    <button class="case" data-case="${i}">
      <span class="case-title">Case ${i + 1}</span>
      <span class="case-meta">${c.age}, ${c.sex.toLowerCase()} · ${c.recordings.length} recordings</span>
    </button>`).join("");
  document.querySelectorAll("[data-case]").forEach(btn => btn.addEventListener("click", () => loadCase(+btn.
dataset.case)));
}

async function loadCase(index) {
  document.querySelectorAll("[data-case]").forEach(b => b.classList.toggle("active", +b.dataset.case === index));
  clearRecordings();
  currentCase = METRICS.demo_cases[index];
  const caseRef = currentCase;
  if (window.innerWidth < 980) $("#verdict-card").scrollIntoView({ behavior: "smooth", block: "start" });
  const items = await Promise.all(caseRef.recordings.map(async r => ({
    buffer: await (await fetch("samples/" + r.file)).arrayBuffer(), site: r.location, name: r.file,
  })));
  if (currentCase !== caseRef) return;
  await addRecordings(items);
}

function clearRecordings() {
```

```
  recordings.forEach(r => URL.revokeObjectURL(r.url));
  recordings = [];
  currentCase = null;
  renderAll();
}

function siteFromName(name) {
  const m = name.match(/(?:^|[_\-\s])(AV|PV|TV|MV|Phc)(?:[_\-\s.]|$)/i);
  if (!m) return null;
  const s = m[1].toUpperCase();
  return s === "PHC" ? "Phc" : s;
}

function setupUpload() {
  const input = $("#file-input");
  const drop = $("#file-drop");
  const handle = async (files) => {
    if (currentCase) clearRecordings();
    const order = ["AV", "PV", "TV", "MV"];
    const used = new Set(recordings.map(r => r.site));
    const items = [];
    for (const file of files) {
      const site = siteFromName(file.name) || order.find(s => !used.has(s)) || "Phc";
      used.add(site);
      items.push({ buffer: await file.arrayBuffer(), site, name: file.name });
    }
    await addRecordings(items);
  };
  input.addEventListener("change", () => { handle([...input.files]); input.value = ""; });
  drop.addEventListener("dragover", e => { e.preventDefault(); drop.classList.add("over"); });
  drop.addEventListener("dragleave", () => drop.classList.remove("over"));
  drop.addEventListener("drop", e => {
    e.preventDefault();
    drop.classList.remove("over");
    handle([...e.dataTransfer.files]);
  });
}

function setupRecorder() {
  const btn = $("#record-btn");
  const status = $("#record-status");
  const siteSelect = $("#record-site");
  siteSelect.addEventListener("change", () => highlightSite(siteSelect.value));

  btn.addEventListener("click", async () => {
    let stream;
    try {
      stream = await navigator.mediaDevices.getUserMedia({
        audio: { echoCancellation: false, noiseSuppression: false, autoGainControl: false },
      });
    } catch {
      status.textContent = "Microphone permission was denied.";
      return;
    }
    if (currentCase) clearRecordings();
    const chunks = [];
    const recorder = new MediaRecorder(stream);
    recorder.ondataavailable = e => chunks.push(e.data);
    recorder.onstop = async () => {
      stream.getTracks().forEach(t => t.stop());
      btn.disabled = false;
      status.textContent = "Analysing...";
      const site = siteSelect.value;
      recordings.filter(r => r.site === site).forEach(r => URL.revokeObjectURL(r.url));
      recordings = recordings.filter(r => r.site !== site);
      await addRecordings([{ buffer: await new Blob(chunks).arrayBuffer(), site, name: `Recording at ${site}` }]);
      status.textContent = "Done. Choose the next site and record again.";
    };
    recorder.start();
    btn.disabled = true;
    let left = 15;
    status.textContent = `Recording... ${left} s`;
    const timer = setInterval(() => {
      left -= 1;
      status.textContent = `Recording... ${left} s`;
      if (left <= 0) {
        clearInterval(timer);
        recorder.stop();
      }
    }, 1000);
  });
}

// ------------------------------------------------------------ rendering

function renderAll() {
  renderChest();
  renderVerdict();
  renderSiteResults();
}

```

```
function siteLabel(site) {
  const done = recordings.filter(r => r.site === site && r.result);
  if (!done.length) return recordings.some(r => r.site === site) ? "busy" : null;
  const labels = done.map(r => LHModel.recordingLabel(r.result));
  if (labels.includes(LHModel.PRESENT)) return LABEL_CLASS[1];
  if (labels.includes(LHModel.UNKNOWN)) return LABEL_CLASS[2];
  return LABEL_CLASS[0];
}

function renderChest() {
  document.querySelectorAll(".site").forEach(g => {
    g.setAttribute("class", "site " + (siteLabel(g.dataset.site) || ""));
  });
}

function highlightSite(site) {
  document.querySelectorAll(".site").forEach(g => g.classList.toggle("focus", g.dataset.site === site));
}

function renderVerdict() {
  const box = $("#verdict");
  const truth = $("#truth");
  const done = recordings.filter(r => r.result);
  const pending = recordings.length - done.length;
  truth.hidden = true;
  $("#verdict-actions").hidden = !done.length || pending > 0;

  if (!recordings.length) {
    box.className = "verdict empty";
    box.innerHTML = verdictHtml("■", "No recordings yet", "Pick a real case or add recordings to start.");
    return;
  }
  if (!done.length) {
    box.className = "verdict empty";
    box.innerHTML = verdictHtml("…", "Analysing", "Turning sound into a spectrogram and running the AI on this device.");
    return;
  }

  const d = LHModel.decide(done.map(r => r.result));
  const trusted = done.filter(r => r.result.beat.ok);
  const strongest = trusted.length ? trusted.reduce((a, b) => (b.result.probs[1] > a.result.probs[1] ? b : a)) : done[0];
  const unclear = done.filter((r, i) => d.labels[i] === LHModel.UNKNOWN).map(r => r.site);
  const noBeat = done.filter(r => !r.result.beat.ok).map(r => r.site);
  const more = pending ? ` Still analysing ${pending} recording${pending > 1 ? "s" : ""}.` : "";

  if (d.label === LHModel.PRESENT) {
    box.className = "verdict bad";
    box.innerHTML = verdictHtml("!", "Murmur suspected: refer for echocardiogram",
      `Strongest at the ${SITE_NAMES[strongest.site].toLowerCase()} area (${pct(d.present)} murmur likelihood). Arrange an echocardiogram to check the heart's structure.${more}`);
  } else if (d.label === LHModel.UNKNOWN && noBeat.length === done.length) {
    box.className = "verdict warn";
    box.innerHTML = verdictHtml("?", "No heartbeat found: check and record again",
      `LittleHeart could not find a regular heartbeat in ${noBeat.length > 1 ? "these recordings" : "this recording"}, so it will not guess. Press the stethoscope firmly on bare skin at the marked spot and keep the room quiet.${more}`);
  } else if (d.label === LHModel.UNKNOWN) {
    box.className = "verdict warn";
    const why = noBeat.length ? ` No heartbeat rhythm was found at ${[...new Set(noBeat)].join(", ")}.` : "";
    box.innerHTML = verdictHtml("?", "Recording unclear: record again",
      `The AI could not judge ${[...new Set(unclear)].join(", ")} reliably.${why} Re-record in a quieter moment; if it stays unclear, refer to a clinician.${more}`);
  } else {
    box.className = "verdict ok";
    box.innerHTML = verdictHtml("✓", "No murmur detected",
      `Highest murmur likelihood ${pct(d.present)} (referral threshold ${pct(d.thresholds.present)}). This does not rule out heart disease; follow usual care.${more}`);
  }

  if (currentCase && !pending) {
    const truthLabel = ["Absent", "Present", "Unknown"].indexOf(currentCase.truth);
    const agree = truthLabel === d.label;
    const details = currentCase.truth === "Present"
      ? `murmur heard at ${currentCase.murmur_locations.join(", ")}${currentCase.timing !== "nan" ? ` · ${currentCase.timing.toLowerCase()}` : ""}${currentCase.grade !== "nan" ? `, grade ${currentCase.grade}` : ""}`
      : currentCase.truth === "Absent" ? "no murmur" : "the expert could not decide (unclear recording)";
    truth.hidden = false;
    truth.className = "truth " + (agree ? "agree" : "disagree");
    truth.innerHTML = `<b>Cardiologist's finding:</b> ${details}. <span>${agree ? "LittleHeart agreed ✓" : "LittleHeart disagreed ✗"}</span>`;
  }
}

function verdictHtml(icon, title, text) {
  return `<div class="verdict-icon" aria-hidden="true">${icon}</div>
    <div><div class="verdict-title">${title}</div><div class="verdict-text">${text}</div></div>`;
}

function renderSiteResults() {
```

```
  const container = $("#site-results");
  const order = { AV: 0, PV: 1, TV: 2, MV: 3, Phc: 4 };
  const sorted = [...recordings].sort((a, b) => order[a.site] - order[b.site] || a.id - b.id);
  const existing = new Map([...container.children].map(el => [+el.dataset.id, el]));

  sorted.forEach(rec => {
    let el = existing.get(rec.id);
    const state = rec.error ? "error" : rec.result ? "done" : "busy";
    if (!el || el.dataset.state !== state) {
      const fresh = buildSiteCard(rec, state);
      if (el) el.replaceWith(fresh); else container.appendChild(fresh);
      el = fresh;
    }
    container.appendChild(el);
    existing.delete(rec.id);
  });
  existing.forEach(el => el.remove());
}

function buildSiteCard(rec, state) {
  const el = document.createElement("div");
  el.className = "card site-card";
  el.dataset.id = rec.id;
  el.dataset.state = state;
  const title = `<div class="site-head"><h3>${SITE_NAMES[rec.site] || rec.site} <span class="site-code">${rec.site}</span></h3>`;

  if (state === "busy") {
    el.innerHTML = `${title}<span class="badge">analysing…</span></div><div class="skeleton"></div>`;
    return el;
  }
  if (state === "error") {
    el.innerHTML = `${title}<span class="badge warn">error</span></div><p class="small">${rec.error}</p>`;
    return el;
  }

  const r = rec.result;
  const t = LHModel.config.thresholds;
  const label = LHModel.recordingLabel(r);
  const badgeText = r.beat.ok ? LABEL_TEXT[label] : "No heartbeat found";
  el.innerHTML = `${title}<span class="badge ${LABEL_CLASS[label]}">${badgeText}</span></div>
    <div class="spec-wrap">
      <canvas class="spec" aria-label="Spectrogram"></canvas>
      <div class="playhead"></div>
      <span class="axis-label top">1000 Hz</span><span class="axis-label bottom">25 Hz</span>
    </div>
    <canvas class="timeline" aria-label="Murmur likelihood over time"></canvas>
    <div class="site-foot">
      <button class="play" aria-label="Play recording">■</button>
      <span class="small muted">${r.duration.toFixed(1)} s · ${r.beat.ok ? "heartbeat ✓" : "no heartbeat rhythm"} · murmur <b>${pct(r.probs[1])}</b> · unclear ${pct(r.probs[2])}</span>
    </div>
    <audio preload="none" src="${rec.url}"></audio>`;

  drawSpectrogram(el.querySelector(".spec"), r.spec);
  requestAnimationFrame(() => drawTimeline(el.querySelector(".timeline"), r, t.present));
  setupPlayback(el, r.duration);
  return el;
}

// ------------------------------------------------------------ drawing

const COLOR_STOPS = [[13, 8, 35], [62, 24, 104], [158, 47, 127], [234, 105, 84], [252, 216, 141]];

function colorAt(v) {
  const x = Math.min(0.9999, Math.max(0, v)) * (COLOR_STOPS.length - 1);
  const i = Math.floor(x), f = x - i;
  return COLOR_STOPS[i].map((c, k) => Math.round(c + (COLOR_STOPS[i + 1][k] - c) * f));
}

function drawSpectrogram(canvas, spec) {
  const nMels = spec.data.length / spec.nFrames;
  canvas.width = spec.nFrames;
  canvas.height = nMels;
  const sorted = Float32Array.from(spec.data).sort();
  const lo = sorted[Math.floor(sorted.length * 0.3)];
  const hi = sorted[Math.floor(sorted.length * 0.995)];
  const ctx = canvas.getContext("2d");
  const img = ctx.createImageData(spec.nFrames, nMels);
  for (let m = 0; m < nMels; m++) {
    for (let f = 0; f < spec.nFrames; f++) {
      const [r, g, b] = colorAt((spec.data[m * spec.nFrames + f] - lo) / (hi - lo));
      const p = ((nMels - 1 - m) * spec.nFrames + f) * 4;
      img.data[p] = r; img.data[p + 1] = g; img.data[p + 2] = b; img.data[p + 3] = 255;
    }
  }
  ctx.putImageData(img, 0, 0);
}

function drawTimeline(canvas, result, threshold) {
  const css = getComputedStyle(document.documentElement);
```

```
  const w = canvas.clientWidth || 600, h = 56;
  const dpr = window.devicePixelRatio || 1;
  canvas.width = w * dpr;
  canvas.height = h * dpr;
  const ctx = canvas.getContext("2d");
  ctx.scale(dpr, dpr);
  const x = (sec) => (sec / result.duration) * w;
  const y = (p) => h - 4 - p * (h - 14);

  ctx.strokeStyle = css.getPropertyValue("--border");
  ctx.setLineDash([3, 3]);
  ctx.beginPath(); ctx.moveTo(0, y(threshold)); ctx.lineTo(w, y(threshold)); ctx.stroke();
  ctx.setLineDash([]);
  ctx.fillStyle = css.getPropertyValue("--muted");
  ctx.font = "10px system-ui, sans-serif";
  ctx.fillText("murmur likelihood per 4 s window · dashed = referral threshold", 4, 10);

  const span = Math.min(4, result.duration);
  result.windows.forEach(win => {
    const p = win.probs[1];
    ctx.fillStyle = p >= threshold ? css.getPropertyValue("--bad") : css.getPropertyValue("--ok");
    ctx.globalAlpha = 0.25 + 0.6 * p;
    const left = x(win.startSec + span / 2 - 1), right = x(Math.min(result.duration, win.startSec + span / 2 +
1));
    ctx.fillRect(left, y(p), Math.max(2, right - left - 1), h - 4 - y(p));
  });
  ctx.globalAlpha = 1;
}

function setupPlayback(el, duration) {
  const audio = el.querySelector("audio");
  const btn = el.querySelector(".play");
  const head = el.querySelector(".playhead");
  let raf = null;
  const tick = () => {
    head.style.left = (100 * audio.currentTime / duration) + "%";
    if (!audio.paused) raf = requestAnimationFrame(tick);
  };
  btn.addEventListener("click", () => {
    document.querySelectorAll("audio").forEach(a => { if (a !== audio) a.pause(); });
    if (audio.paused) audio.play(); else audio.pause();
  });
  audio.addEventListener("play", () => { btn.textContent = "▮▮"; head.style.display = "block"; tick(); });
  audio.addEventListener("pause", () => { btn.textContent = "■"; cancelAnimationFrame(raf); });
  audio.addEventListener("ended", () => { btn.textContent = "■"; head.style.display = "none"; });
}

// ------------------------------------------------------------ validation tab

function renderValidation() {
  const m = METRICS, c = m.cnn, ds = m.dataset;
  const pm = (k) => `${pct(c[k].mean)} <small>± ${Math.round(c[k].std * 100)}</small>`;
  $("#val-intro").textContent =
    `LittleHeart was tested with ${m.folds}-fold cross-validation on ${ds.cv_children} children (${ds.recordings.
toLocaleString("en-US")} recordings in the full dataset). Each child was judged by a model that had never heard
them.`;

  const cards = [
    [c.weighted_accuracy.mean.toFixed(2) + ` <small>± ${c.weighted_accuracy.std.toFixed(2)}</small>`, "Challenge
score",
      "PhysioNet 2022 weighted accuracy: murmurs count 5×, unclear 3×, normal 1×."],
    [pm("sensitivity"), "Murmurs caught", "Children with a murmur that LittleHeart referred."],
    [pm("specificity"), "Normal hearts cleared", "Children without a murmur that were not referred."],
    [c.auc.mean.toFixed(2) + ` <small>± ${c.auc.std.toFixed(2)}</small>`, "AUC", "How well murmur likelihood
ranks children with and without a murmur (1.0 = perfect)."],
  ];
  $("#val-cards").innerHTML = cards.map(([big, title, text]) =>
    `<div class="card metric"><div class="metric-big">${big}</div><h3>${title}</h3><p class="muted small">${
text}</p></div>`).join("");

  const bars = [
    ["Simple baseline (ours)", m.baseline.weighted_accuracy.mean, "base"],
    ["LittleHeart CNN (ours)", c.weighted_accuracy.mean, "ours"],
    ["Best team, PhysioNet 2022*", m.challenge_reference.top_hidden_test, "ref"],
  ];
  $("#compare").innerHTML = bars.map(([name, v, cls]) => `
    <div class="cmp-row"><span class="cmp-name">${name}</span>
      <span class="cmp-track"><span class="cmp-bar ${cls}" style="width:${v * 100}%"></span></span>
      <span class="cmp-val">${v.toFixed(2)}</span></div>`).join("");
  $("#compare-note").textContent =
    "*Not a like-for-like comparison. The challenge's best score (team HearHeart, among 40 ranked murmur entries)
was measured on a hidden test set we cannot access, and scores on that set are usually lower than cross-
validation on the public data. It shows we are in the right range, not that we match the winner.";

  const names = ["No murmur", "Murmur", "Unclear"];
  const cm = m.confusion;
  let html = `<div></div>${names.map(n => `<div class="cm-head">${n}</div>`).join("")}`;
  cm.forEach((row, t) => {
    html += `<div class="cm-side">${names[t]}</div>`;
    row.forEach((v, p) => { html += `<div class="cm-cell ${t === p ? "good" : "bad"}">${v}</div>`; });
  });
```

```
  $("#confusion").innerHTML = html;
  $("#miss-rate").textContent = Math.max(2, Math.round(1 / Math.max(0.01, 1 - c.sensitivity.mean)));
}

function renderSafety() {
  const b = LHModel.beatCheck;
  const rows = [[`Real heart recordings (${b.real_recordings.toLocaleString("en-US")})`, `${(b.real_pass_rate * 100).toFixed(1)}% accepted`, true]];
  Object.entries(b.negatives).forEach(([name, v]) =>
    rows.push([name[0].toUpperCase() + name.slice(1) + " (synthetic)", `${v.rejected}/${v.trials} rejected`, v.rejected === v.trials]));
  $("#safety").innerHTML = rows.map(([name, result, good]) =>
    `<div class="safety-row"><span>${name}</span><b class="${good ? "ok-text" : "warn-text"}">${result}</b></div>`).join("");
}

// ------------------------------------------------------------ start

setupTabs();
setupUpload();
setupRecorder();
renderDemoCases();
renderValidation();
renderAll();
$("#print-btn").addEventListener("click", () => {
  $("#print-meta").textContent = `LittleHeart screening note · ${new Date().toLocaleString("en-GB")} · ` +
    `${recordings.length} recording(s) · Research prototype, not a diagnosis. Referral decisions remain with a clinician.`;
  window.print();
});
$("#clear-btn").addEventListener("click", () => {
  document.querySelectorAll("[data-case]").forEach(b => b.classList.remove("active"));
  clearRecordings();
});
document.querySelectorAll(".site").forEach(g => g.addEventListener("click", () => {
  $("#record-site").value = g.dataset.site;
  highlightSite(g.dataset.site);
}));
const caseFromUrl = parseInt(new URLSearchParams(location.search).get("case"), 10);
if (caseFromUrl >= 1 && caseFromUrl <= METRICS.demo_cases.length) loadCase(caseFromUrl - 1);
LHModel.load().then(renderSafety).catch(err => {
  console.error(err);
  $("#verdict").innerHTML = verdictHtml("!", "AI model failed to load",
    "Open this page through a web server (see README), not by double-clicking the file.");
});
```

web/css/style.css

Visual design, light/dark themes and phone layout.

```
:root {
  --bg: #f7f5f3;
  --surface: #ffffff;
  --surface-2: #f2efec;
  --text: #1d1a1c;
  --muted: #675f63;
  --border: #e6e0dc;
  --accent: #c2364f;
  --accent-soft: #fbe9ec;
  --ok: #2e8f62;
  --ok-soft: #e4f4ec;
  --warn: #b7791f;
  --warn-soft: #fbf1df;
  --bad: #c62f3f;
  --bad-soft: #fbe7e8;
  --torso: #f1e6e0;
  --shadow: 0 1px 2px rgba(30, 20, 25, .05), 0 6px 20px rgba(30, 20, 25, .05);
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --bg: #141113;
    --surface: #1e1a1d;
    --surface-2: #282326;
    --text: #efe9ec;
    --muted: #a99fa4;
    --border: #3a3337;
    --accent: #f06b82;
    --accent-soft: #3a1f26;
    --ok: #4cc38a;
    --ok-soft: #173326;
    --warn: #e9b04f;
    --warn-soft: #3a2e17;
    --bad: #f0606a;
    --bad-soft: #3d1c20;
    --torso: #2c2528;
    --shadow: none;
  }
}

* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font: 16px/1.55 system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}
.wrap { max-width: 1160px; margin: 0 auto; padding: 0 16px; }
h1 { font-size: clamp(1.7rem, 3.6vw, 2.4rem); line-height: 1.15; margin: 0 0 .6rem; letter-spacing: -.01em; }
h2 { font-size: 1.05rem; margin: 0 0 .7rem; }
h3 { font-size: .98rem; margin: 0 0 .4rem; }
p { margin: 0 0 .75rem; }
code { background: var(--surface-2); padding: 1px 5px; border-radius: 4px; font-size: .85em; }
.muted { color: var(--muted); }
.small { font-size: .85rem; }
.mt { margin-top: 1.4rem; }
.ok-text { color: var(--ok); }
.warn-text { color: var(--warn); }
.bad-text { color: var(--bad); }

/* header */
.site-header { background: var(--surface); border-bottom: 1px solid var(--border); position: sticky; top: 0; z-
index: 10; }
.header-row { display: flex; align-items: center; justify-content: space-between; gap: 12px; min-height: 60px;
flex-wrap: wrap; }
.brand { display: flex; align-items: center; gap: 10px; font-weight: 750; font-size: 1.15rem; }
.logo { width: 28px; height: 28px; fill: var(--accent); }
.logo-line { fill: none; stroke: var(--surface); stroke-width: 2; stroke-linejoin: round; stroke-linecap: round; }
.tabs { display: flex; gap: 4px; overflow-x: auto; }
.tab { border: 0; background: none; color: var(--muted); font: inherit; font-size: .95rem; padding: 8px 12px;
border-radius: 8px; cursor: pointer; white-space: nowrap; }
.tab:hover { color: var(--text); }
.tab.active { background: var(--accent-soft); color: var(--accent); font-weight: 650; }

/* layout */
.panel { display: none; padding: 28px 0 8px; }
.panel.active { display: block; }
.intro { max-width: 760px; margin-bottom: 24px; }
.intro p { color: var(--muted); font-size: 1.05rem; }
.card { background: var(--surface); border: 1px solid var(--border); border-radius: 14px; padding: 20px; box-
shadow: var(--shadow); min-width: 0; }
.grid { display: grid; gap: 20px; margin-bottom: 20px; }
.grid.two { grid-template-columns: 1fr 1fr; }
.grid.three { grid-template-columns: repeat(3, 1fr); }
.grid.four { grid-template-columns: repeat(4, 1fr); }
.screen-grid { display: grid; grid-template-columns: 380px 1fr; gap: 20px; align-items: start; }
.results-col { display: grid; gap: 16px; min-width: 0; }
```

```
 84
 85  /* input card */
 86  .source-tabs { display: flex; background: var(--surface-2); border-radius: 10px; padding: 3px; margin-bottom:
     12px; }
 87  .source { flex: 1; border: 0; background: none; font: inherit; font-size: .9rem; color: var(--muted); padding:
     7px; border-radius: 8px; cursor: pointer; }
 88  .source.active { background: var(--surface); color: var(--text); font-weight: 650; box-shadow: 0 1px 2px rgba(0,0,
     0,.08); }
 89  .source-panel { display: none; }
 90  .source-panel.active { display: block; }
 91  .demo-cases { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
 92  .case { text-align: left; border: 1px solid var(--border); background: var(--surface); color: var(--text); border-
     radius: 10px; padding: 8px 10px; cursor: pointer; font: inherit; }
 93  .case:hover { border-color: var(--accent); }
 94  .case.active { border-color: var(--accent); background: var(--accent-soft); }
 95  .case-title { display: block; font-weight: 650; font-size: .92rem; }
 96  .case-meta { display: block; font-size: .78rem; color: var(--muted); }
 97  .file-drop { display: grid; place-items: center; border: 2px dashed var(--border); border-radius: 12px; padding:
     26px 12px; cursor: pointer; text-align: center; color: var(--muted); }
 98  .file-drop.over, .file-drop:hover { border-color: var(--accent); color: var(--text); }
 99  .file-drop input { display: none; }
100  .record-row { display: flex; gap: 8px; }
101  select { font: inherit; color: var(--text); background: var(--surface); border: 1px solid var(--border); border-
     radius: 8px; padding: 8px; flex: 1; min-width: 0; }
102  .btn { font: inherit; font-weight: 650; border: 0; border-radius: 10px; padding: 9px 14px; background: var(--
     accent); color: #fff; cursor: pointer; white-space: nowrap; }
103  .btn:disabled { opacity: .6; cursor: default; }
104
105  /* chest */
106  .chest-wrap { display: grid; grid-template-columns: 170px 1fr; gap: 12px; align-items: center; }
107  .chest { width: 100%; }
108  .torso { fill: var(--torso); }
109  .sternum { stroke: var(--border); stroke-width: 6; stroke-linecap: round; }
110  .heart-shape { fill: var(--accent-soft); stroke: var(--accent); stroke-width: 1; opacity: .8; }
111  .site { cursor: pointer; }
112  .site circle { fill: var(--surface); stroke: var(--muted); stroke-width: 2; transition: fill .3s, stroke .3s; }
113  .site text { font-size: 9px; font-weight: 700; text-anchor: middle; dominant-baseline: central; fill: var(--text);
      pointer-events: none; }
114  .site.focus circle { stroke: var(--accent); stroke-width: 3; }
115  .site.ok circle { fill: var(--ok-soft); stroke: var(--ok); }
116  .site.bad circle { fill: var(--bad-soft); stroke: var(--bad); }
117  .site.warn circle { fill: var(--warn-soft); stroke: var(--warn); }
118  .site.busy circle { stroke: var(--accent); stroke-dasharray: 4 3; animation: spin 1.2s linear infinite; transform-
     origin: center; transform-box: fill-box; }
119  @keyframes spin { to { transform: rotate(360deg); } }
120  .site-legend { list-style: none; padding: 0; margin: 0; display: grid; gap: 6px; font-size: .8rem; color: var(--
     muted); }
121  .site-legend b { color: var(--text); display: inline-block; width: 26px; }
122
123  /* verdict */
124  .verdict-card { scroll-margin-top: 76px; }
125  .verdict { display: flex; gap: 14px; align-items: center; border-radius: 12px; padding: 14px; background: var(--
     surface-2); }
126  .verdict-icon { flex: none; width: 46px; height: 46px; border-radius: 50%; display: grid; place-items: center;
     font-size: 1.4rem; font-weight: 800; background: var(--surface); }
127  .verdict-title { font-weight: 750; font-size: 1.12rem; line-height: 1.3; }
128  .verdict-text { color: var(--muted); font-size: .92rem; margin-top: 2px; }
129  .verdict.ok { background: var(--ok-soft); } .verdict.ok .verdict-icon { color: var(--ok); }
130  .verdict.bad { background: var(--bad-soft); } .verdict.bad .verdict-icon { color: var(--bad); }
131  .verdict.warn { background: var(--warn-soft); } .verdict.warn .verdict-icon { color: var(--warn); }
132  .verdict-actions { display: flex; gap: 8px; margin-top: 12px; flex-wrap: wrap; }
133  .verdict-actions[hidden] { display: none; }
134  .btn-ghost { font: inherit; font-size: .88rem; font-weight: 600; border: 1px solid var(--border); background: var(
     --surface); color: var(--text); border-radius: 9px; padding: 7px 12px; cursor: pointer; }
135  .btn-ghost:hover { border-color: var(--accent); color: var(--accent); }
136  .print-only { display: none; }
137  .truth { margin-top: 12px; font-size: .9rem; padding: 10px 12px; border-radius: 10px; border: 1px solid var(--
     border); }
138  .truth span { font-weight: 700; margin-left: 4px; }
139  .truth.agree span { color: var(--ok); }
140  .truth.disagree span { color: var(--bad); }
141
142  /* site cards */
143  .site-results { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
144  .site-card { padding: 14px; }
145  .site-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px; }
146  .site-head h3 { margin: 0; }
147  .site-code { color: var(--muted); font-weight: 500; font-size: .8rem; }
148  .badge { font-size: .75rem; font-weight: 700; padding: 3px 9px; border-radius: 99px; background: var(--surface-2);
      color: var(--muted); }
149  .badge.ok { background: var(--ok-soft); color: var(--ok); }
150  .badge.bad { background: var(--bad-soft); color: var(--bad); }
151  .badge.warn { background: var(--warn-soft); color: var(--warn); }
152  .spec-wrap { position: relative; border-radius: 8px; overflow: hidden; }
153  .spec { display: block; width: 100%; height: 110px; image-rendering: auto; }
154  .playhead { display: none; position: absolute; top: 0; bottom: 0; width: 2px; background: #fff; box-shadow: 0 0
     4px rgba(0,0,0,.6); }
155  .axis-label { position: absolute; left: 6px; font-size: 10px; color: rgba(255,255,255,.75); }
156  .axis-label.top { top: 3px; } .axis-label.bottom { bottom: 3px; }
157  .timeline { display: block; width: 100%; height: 56px; margin-top: 4px; }
158  .site-foot { display: flex; align-items: center; gap: 10px; margin-top: 6px; }
```

```
.play { width: 32px; height: 32px; border-radius: 50%; border: 1px solid var(--border); background: var(--surface); color: var(--text); cursor: pointer; font-size: .8rem; flex: none; }
.play:hover { border-color: var(--accent); color: var(--accent); }
.skeleton { height: 170px; border-radius: 8px; background: linear-gradient(90deg, var(--surface-2), var(--border), var(--surface-2)); background-size: 200% 100%; animation: shimmer 1.2s infinite; }
@keyframes shimmer { to { background-position: -200% 0; } }

.disclaimer { margin-top: 24px; font-size: .85rem; color: var(--muted); border-left: 3px solid var(--warn); padding: 8px 12px; background: var(--surface); border-radius: 6px; }

/* validation */
.metric { text-align: center; }
.metric-big { font-size: 2rem; font-weight: 800; color: var(--accent); line-height: 1.2; }
.metric-big small { font-size: .9rem; font-weight: 600; color: var(--muted); }
.compare { display: grid; gap: 12px; margin: 8px 0 12px; }
.cmp-row { display: grid; grid-template-columns: 190px 1fr 44px; gap: 10px; align-items: center; font-size: .9rem; }
.cmp-track { height: 14px; background: var(--surface-2); border-radius: 7px; overflow: hidden; }
.cmp-bar { display: block; height: 100%; border-radius: 7px; }
.cmp-bar.base { background: var(--muted); }
.cmp-bar.ours { background: var(--accent); }
.cmp-bar.ref { background: var(--border); background-image: repeating-linear-gradient(45deg, transparent 0 5px, var(--surface) 5px 8px); border: 1px solid var(--muted); }
.cmp-val { font-weight: 700; font-variant-numeric: tabular-nums; }
.confusion { display: grid; grid-template-columns: auto repeat(3, 1fr); gap: 6px; font-size: .85rem; }
.cm-head { text-align: center; color: var(--muted); font-weight: 650; }
.cm-side { color: var(--muted); font-weight: 650; display: flex; align-items: center; }
.cm-cell { text-align: center; padding: 12px 4px; border-radius: 8px; font-weight: 750; font-size: 1.2rem; font-variant-numeric: tabular-nums; }
.cm-cell.good { background: var(--ok-soft); }
.cm-cell.bad { background: var(--bad-soft); }
.steps, .checks { margin: 0; padding-left: 20px; display: grid; gap: 8px; font-size: .93rem; }
.safety-card { margin-bottom: 20px; }
.safety { display: grid; grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); gap: 8px 24px; }
.safety-row { display: flex; justify-content: space-between; gap: 12px; padding: 6px 0; border-bottom: 1px solid var(--border); font-size: .9rem; }
.status-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 20px; }
.status-grid ul { margin: 0; padding-left: 18px; display: grid; gap: 6px; font-size: .9rem; }

.footer { color: var(--muted); font-size: .85rem; padding: 28px 16px 40px; }

@media print {
  .site-header, .intro, .input-card, .verdict-actions, .play, .footer, .truth { display: none !important; }
  .print-only { display: block; margin-top: 10px; }
  body { background: #fff; }
  .screen-grid { grid-template-columns: 1fr; }
  .site-results { grid-template-columns: 1fr 1fr; }
  .card { box-shadow: none; break-inside: avoid; }
  .spec, .timeline { print-color-adjust: exact; -webkit-print-color-adjust: exact; }
}
@media (max-width: 980px) {
  .screen-grid { grid-template-columns: 1fr; }
  .grid.four { grid-template-columns: 1fr 1fr; }
  .grid.three { grid-template-columns: 1fr 1fr; }
}
@media (max-width: 680px) {
  .grid.two, .grid.three, .status-grid, .site-results { grid-template-columns: 1fr; }
  .cmp-row { grid-template-columns: 1fr 44px; }
  .cmp-name { grid-column: 1 / -1; }
  .chest-wrap { grid-template-columns: 120px 1fr; }
}
@media (max-width: 420px) {
  .grid.four { grid-template-columns: 1fr; }
  .case-meta { font-size: .72rem; }
}
```

tests/test_pipeline.py

Automated tests, including Python vs JavaScript feature parity.

```
"""Run with:  python -m pytest tests"""

import json
import shutil
import subprocess
import sys
from pathlib import Path

import numpy as np
import pytest

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "ml"))

import features as F  # noqa: E402
from heartbeat_check import beat_measures  # noqa: E402
from train import ABSENT, PRESENT, UNKNOWN, decide, weighted_accuracy  # noqa: E402

SAMPLES = sorted((ROOT / "web" / "samples").glob("*.wav"))
CONFIG = ROOT / "web" / "model" / "config.json"
CHECK = ROOT / "web" / "model" / "heartbeat_check.json"


def test_mel_filters_cover_range_in_order():
    fb = F.MEL_FB
    assert fb.shape == (F.N_MELS, F.N_FFT // 2 + 1)
    peaks = fb.argmax(axis=1)
    assert np.all(np.diff(peaks) >= 0), "filters should go from low to high pitch"
    assert np.all(fb.max(axis=1) > 0), "every filter should catch some frequency"


def test_pure_tone_lands_in_matching_band():
    t = np.arange(F.SAMPLE_RATE * 2) / F.SAMPLE_RATE
    for freq in (60, 200, 600):
        spec = F.log_mel(np.sin(2 * np.pi * freq * t))
        band = spec.mean(axis=1).argmax()
        centers = F.mel_to_hz(np.linspace(F.hz_to_mel(F.F_MIN), F.hz_to_mel(F.F_MAX), F.N_MELS + 2))[1:-1]
        assert abs(centers[band] - freq) < 40, f"{freq} Hz tone went to band centred at {centers[band]:.0f} Hz"


def test_loudness_does_not_change_spectrogram():
    rng = np.random.default_rng(0)
    x = rng.normal(size=8000)
    assert np.allclose(F.log_mel(x), F.log_mel(x * 7.5), atol=1e-4)


def test_short_recording_is_repeated_to_fill_window():
    spec = np.arange(F.N_MELS * 100, dtype=np.float32).reshape(F.N_MELS, 100)
    w = F.cut_window(spec, 0)
    assert w.shape == (F.N_MELS, F.WINDOW_FRAMES)
    assert np.array_equal(w[:, 100:200], spec)


def test_weighted_accuracy_matches_challenge_definition():
    # 2 present (1 correct), 1 unknown (correct), 3 absent (2 correct)
    y_true = [PRESENT, PRESENT, UNKNOWN, ABSENT, ABSENT, ABSENT]
    y_pred = [PRESENT, ABSENT, UNKNOWN, ABSENT, ABSENT, PRESENT]
    expected = (5 * 1 + 3 * 1 + 1 * 2) / (5 * 2 + 3 * 1 + 1 * 3)
    assert weighted_accuracy(y_true, y_pred) == pytest.approx(expected)


def test_decision_rule_order():
    probs = np.array([[0.9, 0.9], [0.1, 0.9], [0.1, 0.1]])  # (present, unknown) per child
    assert list(decide(probs, 0.5, 0.5)) == [PRESENT, UNKNOWN, ABSENT]


@pytest.mark.skipif(not SAMPLES or not CONFIG.exists(), reason="run ml/train.py first")
@pytest.mark.skipif(shutil.which("node") is None, reason="Node.js not installed")
def test_javascript_features_match_python():
    for wav in SAMPLES[:3]:
        out = subprocess.run(["node", str(ROOT / "tests" / "js_logmel.cjs"), str(wav), str(CONFIG)],
                             capture_output=True, text=True, check=True)
        js = json.loads(out.stdout)
        py = F.log_mel(F.load_wav(wav))
        assert js["nFrames"] == py.shape[1]
        assert js["starts"] == F.window_starts(py.shape[1])
        diff = np.abs(np.array(js["data"]).reshape(py.shape) - py).max()
        assert diff < 1e-3, f"{wav.name}: max difference {diff}"
        rhythm, depth = beat_measures(py)
        assert js["beat"]["rhythm"] == pytest.approx(rhythm, abs=1e-3)
        assert js["beat"]["depth"] == pytest.approx(depth, abs=1e-3)


@pytest.mark.skipif(not CHECK.exists(), reason="run ml/heartbeat_check.py first")
def test_heartbeat_check_rejects_non_heart_sounds_and_accepts_demo_cases():
    limits = json.loads(CHECK.read_text())
    passes = lambda m: m[0] >= limits["rhythm_min"] and m[1] >= limits["depth_min"]
```

```
    rng = np.random.default_rng(123)
    t = np.arange(F.SAMPLE_RATE * 10) / F.SAMPLE_RATE
    assert not passes(beat_measures(F.log_mel(rng.normal(size=len(t)))))
    assert not passes(beat_measures(F.log_mel(np.sin(2 * np.pi * 440 * t))))
    accepted = [passes(beat_measures(F.log_mel(F.load_wav(w)))) for w in SAMPLES]
    assert np.mean(accepted) >= 0.9


@pytest.mark.skipif(not CONFIG.exists(), reason="run ml/train.py first")
def test_exported_model_outputs_probabilities():
    ort = pytest.importorskip("onnxruntime")
    session = ort.InferenceSession(str(ROOT / "web" / "model" / "murmur_cnn.onnx"))
    x = np.random.default_rng(1).normal(-1, 1, (3, 1, F.N_MELS, F.WINDOW_FRAMES)).astype(np.float32)
    probs = session.run(None, {"logmel": x})[0]
    assert probs.shape == (3, 3)
    assert np.allclose(probs.sum(axis=1), 1, atol=1e-5)
```

tests/js_logmel.cjs

Node.js helper used by the parity test.

```
// Helper for tests/test_pipeline.py: prints the JavaScript log-mel of a WAV file as JSON.
// Usage: node tests/js_logmel.cjs <file.wav> <config.json>
const fs = require("fs");
const path = require("path");
const F = require(path.join(__dirname, "..", "web", "js", "features.js"));

const [wavPath, configPath] = process.argv.slice(2);
const buf = fs.readFileSync(wavPath);
const wav = F.parseWav(buf.buffer.slice(buf.byteOffset, buf.byteOffset + buf.byteLength));
const cfg = JSON.parse(fs.readFileSync(configPath, "utf8")).features;
const spec = F.logMel(wav.samples, cfg);
const starts = F.windowStarts(spec.nFrames, cfg);
const check = JSON.parse(fs.readFileSync(path.join(path.dirname(configPath), "heartbeat_check.json"), "utf8"));
const beat = F.beatMeasures(spec, check);
process.stdout.write(JSON.stringify({ nFrames: spec.nFrames, data: Array.from(spec.data), starts, beat }));
```

docs/make_pdfs.py

Generates this PDF and the one-page description.

```
"""
Builds the two PDFs required for the UnivaBio submission:

  docs/LittleHeart_Project_Description.pdf   one page
  docs/LittleHeart_Code.pdf                  all source code with line numbers

Run after training:  python docs/make_pdfs.py
"""

import json
from pathlib import Path

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import mm
from reportlab.platypus import (Image, PageBreak, Paragraph, Preformatted,
                                SimpleDocTemplate, Spacer, Table, TableStyle)

ROOT = Path(__file__).resolve().parent.parent
DOCS = ROOT / "docs"
ACCENT = colors.HexColor("#c2364f")
GREY = colors.HexColor("#675f63")

CODE_FILES = [
    ("ml/download_data.py", "Downloads the CirCor DigiScope dataset from PhysioNet's open-data mirror."),
    ("ml/features.py", "Turns a recording into a log-mel spectrogram and 4-second windows."),
    ("ml/train.py", "Cross-validated training of the CNN and baseline, ONNX export, demo cases."),
    ("ml/heartbeat_check.py", "Safety check calibration: refuse recordings without a heartbeat rhythm."),
    ("web/index.html", "Page structure: screening, validation and education tabs."),
    ("web/js/features.js", "JavaScript port of features.py, so the browser computes identical spectrograms."),
    ("web/js/model.js", "Runs the ONNX model in the browser and applies the referral rule."),
    ("web/js/app.js", "User interface: cases, uploads, microphone, spectrograms, results, validation charts."),
    ("web/css/style.css", "Visual design, light/dark themes and phone layout."),
    ("tests/test_pipeline.py", "Automated tests, including Python vs JavaScript feature parity."),
    ("tests/js_logmel.cjs", "Node.js helper used by the parity test."),
    ("docs/make_pdfs.py", "Generates this PDF and the one-page description."),
]


def styles():
    base = getSampleStyleSheet()
    return {
        "title": ParagraphStyle("t", parent=base["Title"], fontSize=21, textColor=ACCENT, spaceAfter=1),
        "tagline": ParagraphStyle("tg", parent=base["Normal"], alignment=TA_CENTER, fontSize=9.5, textColor=GREY,
spaceAfter=5),
        "h": ParagraphStyle("h", parent=base["Heading2"], fontSize=10.2, textColor=ACCENT, spaceBefore=4,
spaceAfter=1),
        "body": ParagraphStyle("b", parent=base["Normal"], fontSize=8.3, leading=10.7),
        "small": ParagraphStyle("s", parent=base["Normal"], fontSize=7.3, leading=9, textColor=GREY),
        "code": ParagraphStyle("c", parent=base["Code"], fontSize=6.8, leading=8.3, leftIndent=0),
    }


def build_description(m):
    s = styles()
    c, b, ds = m["cnn"], m["baseline"], m["dataset"]
    pct = lambda x: f"{round(x * 100)}%"

    left = [
        Paragraph("The problem", s["h"]),
        Paragraph("About 1 in 100 babies is born with a heart defect, and rheumatic heart disease affects tens of
millions of "
                  "people, most of them young and living in low- and middle-income countries. Both are often
first noticed as a heart murmur. "
                  "But recognising a murmur through a stethoscope takes years of training, and the echocardiogram
that confirms "
                  "it, and the cardiologists who read it, are scarce exactly where these diseases are most common.
 Many children "
                  "are diagnosed only when the heart is already damaged.", s["body"]),
        Paragraph("Our solution", s["h"]),
        Paragraph("<b>LittleHeart</b> is a web app that lets a community health worker, school nurse or GP record
a child's "
                  "heart at the four standard valve areas and instantly learn whether a murmur is likely:
<i>refer for "
                  "echocardiogram</i>, <i>no murmur detected</i>, or <i>recording unclear, record again</i>. It
turns an "
                  "expert skill into a consistent second opinion so scarce echocardiograms go to the children who
need them.", s["body"]),
        Paragraph("Key features", s["h"]),
        Paragraph("• <b>Three ways in</b>: real held-out patient cases, file upload, or live microphone/
stethoscope recording.<br/>"
                  "• <b>Per-valve results</b> on an interactive chest diagram.<br/>"
                  "• <b>Explainable</b>: spectrogram of each recording with a moving playhead and a murmur-
likelihood timeline.<br/>"
                  "• <b>Private</b>: the AI runs in the browser (ONNX Runtime Web); no audio is uploaded.<br/>"
```

```
                  "• <b>Transparent validation tab</b> and an honest \"working / not yet / limits\" table.<br/>"
                  "• Education tab on murmurs, recording technique and next steps; phone layout and dark mode.", s["body"]),
    ]
    right = [
        Paragraph("How the AI works", s["h"]),
        Paragraph(f"Trained on the CirCor DigiScope dataset (PhysioNet): {ds['recordings']:,} recordings from {ds['children']} "
                  "children in Brazil, labelled by cardiologists. Each recording becomes a log-mel spectrogram (25-1000 Hz), "
                  "cut into 4-second windows. A small convolutional neural network (about 61,000 weights, PyTorch) scores each "
                  "window as murmur / no murmur / unclear; scores are averaged per valve, and a child is referred if any valve "
                  "crosses a tuned threshold. A <b>heartbeat safety check</b> first refuses recordings with no 40-220 bpm "
                  "rhythm (noise, tones, most speech), so the AI never guesses on sounds it was not trained for. The same "
                  "code exists in Python and JavaScript and automated tests check they match.", s["body"]),
        Paragraph("Results", s["h"]),
        Paragraph(f"{m['folds']}-fold cross-validation, split by child, on {ds['cv_children']} children: official PhysioNet 2022 "
                  f"murmur score <b>{c['weighted_accuracy']['mean']:.2f} ± {c['weighted_accuracy']['std']:.2f}</b> "
                  f"(simple baseline {b['weighted_accuracy']['mean']:.2f}; for context, the best challenge team scored 0.78 on a "
                  f"harder hidden test set, so this is not a like-for-like comparison). "
                  f"It caught <b>{pct(c['sensitivity']['mean'])}</b> of children with murmurs and cleared "
                  f"<b>{pct(c['specificity']['mean'])}</b> of those without (AUC {c['auc']['mean']:.2f}). "
                  f"{ds['demo_children']} demo children were never used in training.", s["body"]),
        Paragraph("Honest limits", s["h"]),
        Paragraph("A screening aid, not a diagnosis or medical device. It still misses some murmurs, was trained on one "
                  "population and one stethoscope, cannot yet tell innocent from pathological murmurs, and has not been "
                  "tested prospectively or with phone microphones. All of this is stated in the app.", s["body"]),
        Paragraph("Next steps", s["h"]),
        Paragraph("Test with low-cost stethoscope adapters, validate on other datasets and populations, add murmur timing and "
                  "grading, multilingual interface, and a pilot with a school RHD-screening programme.", s["body"]),
    ]

    story = [Paragraph("LittleHeart", s["title"]),
             Paragraph("AI heart murmur screening for children · UnivaBio 2026: AI for Human Health", s["tagline"])]
    cols = Table([[left, right]], colWidths=[92 * mm, 92 * mm])
    cols.setStyle(TableStyle([("VALIGN", (0, 0), (-1, -1), "TOP"), ("LEFTPADDING", (0, 0), (-1, -1), 0),
                              ("RIGHTPADDING", (0, 0), (0, 0), 4 * mm)]))
    story += [cols, Spacer(1, 2.5 * mm)]
    shot = DOCS / "screenshot.png"
    if shot.exists():
        img = Image(str(shot))
        img._restrictSize(165 * mm, 98 * mm)
        story.append(img)
    story += [Spacer(1, 1.5 * mm),
              Paragraph("Tech: Python, PyTorch, scikit-learn, NumPy (training) · HTML, CSS, JavaScript, ONNX Runtime Web (app) · "
                        "Data: CirCor DigiScope Phonocardiogram Dataset, Oliveira et al. 2022, PhysioNet, CC BY 4.0.", s["small"])]

    doc = SimpleDocTemplate(str(DOCS / "LittleHeart_Project_Description.pdf"), pagesize=A4, leftMargin=13 * mm,
                            rightMargin=13 * mm, topMargin=10 * mm, bottomMargin=9 * mm, title="LittleHeart - Project Description")
    doc.build(story)
    return doc


def build_code_pdf():
    s = styles()
    story = [Paragraph("LittleHeart: source code", s["title"]),
             Paragraph("UnivaBio 2026 · AI for Human Health", s["tagline"]), Spacer(1, 4 * mm),
             Paragraph("Project structure", s["h"])]
    for path, purpose in CODE_FILES:
        story.append(Paragraph(f"<b>{path}</b>: {purpose}", s["body"]))
    story.append(Paragraph("Generated by ml/train.py (not listed): web/model/murmur_cnn.onnx (trained network), "
                           "config.json (spectrogram settings and thresholds), metrics.json/.js (validation results), "
                           "web/samples/*.wav (demo recordings from the dataset).", s["body"]))
    for path, purpose in CODE_FILES:
        story += [PageBreak(), Paragraph(path, s["h"]), Paragraph(purpose, s["small"]), Spacer(1, 2 * mm)]
        lines = (ROOT / path).read_text(encoding="utf-8").splitlines()
        story.append(Preformatted("\n".join(f"{i:4d}  {line}" for i, line in enumerate(lines, 1)),
                                  s["code"], maxLineLength=120, newLineChars="      "))

    def footer(canvas, doc):
        canvas.setFont("Helvetica", 7)
        canvas.setFillColor(colors.grey)
        canvas.drawRightString(A4[0] - 13 * mm, 8 * mm, f"LittleHeart · page {doc.page}")

    doc = SimpleDocTemplate(str(DOCS / "LittleHeart_Code.pdf"), pagesize=A4, leftMargin=13 * mm, rightMargin=13 *
```

```
    mm,
                                topMargin=12 * mm, bottomMargin=14 * mm, title="LittleHeart - Source Code")
    doc.build(story, onFirstPage=footer, onLaterPages=footer)
    return doc


if __name__ == "__main__":
    metrics = json.loads((ROOT / "web" / "model" / "metrics.json").read_text())
    d = build_description(metrics)
    c = build_code_pdf()
    print(f"Description: {d.page} page(s); code listing: {c.page} page(s)")
```